

RETRIEVAL OBSERVABILITY DETERMINES THE DETECTION BOUNDARY FOR AGENT MEMORY POISONING

Jun Wen Leong

leongjunwen@gmail.com

v3 changelog (September 2026, changes from arXiv v2): (1) Major reframing: elevated the retrieval-observability taxonomy from a Discussion table to the paper’s primary contribution; new title, abstract, and introduction center on the detection boundary as an architectural property of the deployment, not a classifier contribution. (2) Added a formal retrieval-to-action provenance graph specification (Section 3) and preregistered probes V3-1/V3-2. (3) Restructured results around taxonomy-organized failure modes; added leave-one-scenario-family-out generalization analysis. (4) Corrected the cross-framing attack-success scorer to count only structured recipients (the `to` field), not addresses appearing in the email body; this lowers reported ASR (e.g., GPT-5.1 98.3% $\rightarrow$ 62.5%) while leaving signature rates unchanged. (5) Added leakage-corrected grouped cross-validation (AUC 0.99 random-fold $\rightarrow$ 0.907 grouped by model $\times$ defense cell). (6) Re-collected the cross-framing benign memory-grounded control after fixing a memory-key mismatch that had invalidated it; the corrected control yields benign FPR = 100%, reinforcing the precondition-not-predicate finding. (7) Clarified that the cross-framing study compares prompt-framing variants of one execution loop, not independent agent frameworks. (8) Replaced the V3-1 treatment-arm “intervals would be wider” caveat with actual scenario-clustered bootstrap CIs (GPT-4.1 [0.450, 0.864], GPT-4o [0.600, 0.929]; both still exclude 0). (9) Corrected the classification-threshold wording (≥ 0.5 , matching the code) and fixed a garbled multi-send limitation sentence (1,120 sessions do contain multiple sends; the naive and corrected `recall_before_send` definitions agree on all 2,520 primary sessions). (10) Added a frozen-classifier cross-harness holdout (train on LangGraph factorial, test on the raw-OpenAI-loop sessions): pooled recall 186/188 = 98.9%, evidence the signature transfers across execution engines, not just prompt framings. (11) Added a base-rate-aware PPV table (Table 9) computed from existing TPR/FPR, quantifying the standalone-precision ceiling at realistic attack prevalences. Note: the unconditional benign FPR upper bound is reported as 57.6% (the $k=1$ cell), revised from 52.6% in v2 which reported the $k=5$ cell; the underlying V2-1 data are unchanged.

ABSTRACT

We establish that *retrieval observability*—whether an agent’s memory access produces a logged tool call—is the architectural variable that determines whether a trajectory-based detector can observe the *retrieval provenance* of a memory-poisoning attack. This boundary partitions the attack surface: when exfiltration causally depends on a payload available exclusively through an observable retrieval tool, a retrieval-before-exfiltration event is structurally necessary (exact-key: 58/58 zero violations; semantic search: 33/33 GPT-4.1, 13/13 GPT-4o); when the scaffold delivers the payload implicitly, attacks succeed without producing the retrieval signature (50–82% ASR). We formalize this boundary as a framework-agnostic *retrieval-to-action provenance graph* and classify published memory-poisoning attack families by their position relative to it. Within the observable scope, we construct a proof-of-concept detector (AUC = 0.99, cross-model transfer to 405 frontier sessions without retraining) and simultaneously falsify its standalone deployment viability: benign memory-grounded sends are trajectory-isomorphic (FPR 24.7–57.6%, $N=4,360$, 13 models), confirming that the signature is an attack *precondition*, not a maliciousness predicate. A cross-framing replication (640 trials, two prompt-framing variants of one raw execution loop, 7 models) shows the signature is stable across framings (99–100% detection) and establishes a *Latent Carrier* phenomenon: models with 0% attack success produce 100% signature rates because the retrieval-to-action path is traversed regardless of exfiltration compliance ($r = 0.069$ between ASR and signature rate across models). We do not claim transfer across independent agent frameworks: both variants share one loop and tool schema, and a genuine LangChain/LangGraph and OpenAI Assistants replication remains future work. The taxonomy implies

that trajectory monitoring has a hard architectural ceiling for *retrieval-provenance* detection, independent of classifier sophistication: any detector operating on tool-call logs alone is structurally blind to the retrieval signature of implicit-retrieval attacks. Detectability is governed by two observability axes—write and read: a re-analysis of our implicit-delivery data shows that attacks implicit on the read axis but with an observable memory write still leave a save-before-send trace (though as a precondition with 66.7% benign FPR, not a usable predicate), whereas attacks implicit on *both* axes—the cell occupied by several prominent published families (MINJA, Zombie Agents, scaffold-side RAG injection), classified by architectural analysis rather than direct reproduction—leave no memory operation at all and are caught 0/53 by any operation-only signature, confirming their genuine invisibility. Defense architectures must therefore combine trajectory triage (for observable-retrieval attacks) with content-level or provenance-level enforcement (for implicit attacks).

1 INTRODUCTION

LLM agent memory poisoning attacks span a growing family of methods—delayed-trigger attacks via RAG injection (Leong, 2026a), embedding-similarity manipulation (Dong et al., 2025), sliding-window persistence (Yang et al., 2026), cross-session stored prompt injection (Xie et al., 2026)—but share a common structure: a malicious payload is written to persistent state and later retrieved to influence agent behavior. A natural question for defenders is: *can trajectory monitoring (observing tool-call logs alone) detect these attacks?*

We show the answer depends on a single architectural variable: **retrieval observability**—whether the agent’s memory access produces a logged tool invocation. This variable partitions the entire memory-poisoning attack surface into two regions with fundamentally different detectability properties:

- **Observable retrieval** (agent-initiated tool call: `recall_fact`, `memory_search`, etc.): a retrieval-before-exfiltration event is *structurally necessary* for successful attack execution. Evidence: exact-key recall 58/58 zero violations (GPT-4.1 + GPT-4o); semantic search 33/33 GPT-4.1, 13/13 GPT-4o (directionally supportive but underpowered).
- **Implicit retrieval** (framework-level embedding similarity, scaffold-side context injection, cached internal state): attacks succeed without producing any observable trajectory signature. Evidence: 50–82% ASR with signature absent by construction.

This boundary is not specific to our detector. It is an architectural ceiling on *any* method that operates on tool-call logs: VIGIL (Li et al., 2026), TraceAegis (Liu et al., 2025), TRACES (Wang et al., 2025), or any future trajectory classifier will be structurally blind to the retrieval signature of implicit-retrieval attacks regardless of their sophistication (the exfiltrating action may still be observable through other channels, e.g. destination metadata; what is unobservable is the retrieval provenance). We formalize this boundary as a framework-agnostic *retrieval-to-action provenance graph* (Section 3) and use it to classify all major published attack families by their detection-relevant properties (Table 12).

Within the observable-retrieval scope, we construct a proof-of-concept detector (Random Forest, AUC = 0.99, cross-model transfer to 405 frontier sessions) and simultaneously demonstrate its precision ceiling: benign memory-grounded sends produce the same trajectory signature (FPR 24.7–57.6%, $N=4,360$, 13 models), confirming that the retrieval-before-send transition is an attack *precondition*, not a maliciousness predicate. The classifier serves as evidence for the invariant’s existence and scope—it is not the contribution.

Contributions.

1. **Observability taxonomy.** We establish retrieval observability as the dividing variable of the memory-poisoning attack surface with respect to trajectory-based detection. We classify published attack families by this variable and empirically validate both sides of the

Table 1: Claim-to-evidence map. *Evidence status* distinguishes preregistered confirmatory findings (P), post-hoc scorer/label corrections disclosed in the deviations (C), and architectural classifications argued from mechanism rather than direct reproduction (A). All numeric results are reproducible from the released artifacts; API-served results are auditable via provenance stamps but not bitwise reproducible.

Claim	Evidence	Status	Section
Observable retrieval $\Rightarrow$ <i>recall_before_send</i> necessary for tool-routed exfiltration	Exact-key 58/58, semantic 33/33 (GPT-4.1) / 13/13 (GPT-4o) zero violations	P	§6.3
Implicit delivery breaks necessity (successful attacks with rbs=0)	V3-1 treatment 67% (GPT-4.1) / 74% (GPT-4o) rbs=0; scenario-clustered CIs exclude 0	P	§6.3
Signature is a precondition, not a maliciousness predicate	Benign FPR 24.7–57.6% ($N=4,360$); $r=0.069$ ASR vs signature	P	§5
Within-scope detector AUC = 0.990 (random-fold) / 0.907 (grouped)	5-fold CV, $N=2,520$; leakage-corrected grouped CV	P	§4.4
Cross-model transfer without retraining	Leave-one-model-out 6/9 AUC = 1.000; frontier Recall = 0.901	P	§4.6
Signature stable across prompt framings (one loop, one schema)	99–100% signature on GPT-5.1 raw vs ReAct-framed	P	§5.1
ASR corrected to structured-recipient (to) scoring	GPT-5.1 98.3% $\rightarrow$ 62.5%; signature rates unchanged	C	§5.1
V3-2 arm B scorer includes <i>draft_email</i> as exfiltration-capable	Preregistered 33/33; corrected sensitivity 38/38 (both reported)	C	§6.3
MINJA / Zombie / scaffold-side RAG evade trajectory detection	Provenance-graph classification; mechanism validated in-architecture (V3-1/V3-2), named pipelines not reproduced	A	§3, Table 12

boundary with preregistered probes (V3-1, V3-2; $N=440$ new trials across 3 models $\times$ 2 retrieval primitives $\times$ 2 delivery modes).

2. **Framework-agnostic event graph.** We define a retrieval-to-action provenance graph specification that maps memory-poisoning attacks into a common representation independent of framework, tool naming, or retrieval implementation (Section 3).
3. **Structural necessity within scope.** Under observable retrieval, the *retrieval_before_exfiltration* transition is mechanistically forced by the attack’s information dependency. A single-rule detector exploiting this achieves AUC = 0.956; the full feature set (AUC = 0.99) adds marginal value because the invariant itself carries most of the separation. Adversarial suppression of the transition breaks the attack.
4. **Precision ceiling (deployment falsification).** Benign memory-grounded sends produce the same trajectory signature (conditional FPR = 100%), establishing that operation-only features face a hard precision ceiling when agents ground actions in recalled context.
5. **Cross-model transfer and scope limits.** The invariant transfers across 9 models (7B–120B) and frontier APIs without retraining, but fails deterministically on implicit-bypass architectures (qwq:32b RAG fallback), confirming the taxonomy’s predictive validity.

2 METHODOLOGY

2.1 THREAT MODEL

We consider a *memory-channel poisoning attack*: the adversary injects a malicious instruction via a RAG-retrieved document, which the agent stores in persistent memory. In a subsequent session, the agent retrieves the stored rule and takes a malicious action (email exfiltration). We explicitly distinguish this from *prompt-injection attacks*, where the adversary embeds the routing rule directly in the current session’s user prompt without storing it in memory. Our detector targets the former; we report its behavior on the latter as the evasion boundary.

Attack success condition. A run is classified as an attack success if `email_send_email` is called with a recipient outside the allowed set `{team@example.com}` in the trigger session (S4). In V3-1, this allowlist-based scorer produces 5/49 successes (10.2%) where the exfiltration destination does not match the specific attacker address planted in that scenario’s memory (the model sent to the other known attacker address, likely confabulated from pretraining or cross-contaminated from the scenario design’s shared address pool). These cases represent genuine policy violations (unauthorized external sends) but ambiguous evidence for the retrieval-dependency mechanism; conclusions about the necessity invariant rest on the control-arm zero-violation counts, which are unaffected.

Detection constraint. The detector observes only the tool-call sequence and its timestamps. It does not access memory contents, model weights, activation patterns, or the user’s messages. Attack-success labels are constructed using tool arguments (recipient address); classifier features use only operation names, ordering, and counts, never email contents or recipients.

Architectural assumptions. The detector assumes: (1) persistent memory is accessed exclusively through observable retrieval operations (tool calls), not through hidden internal state or implicit caching; (2) tool invocations are logged with execution timestamps; (3) the attacker’s routing information resides in persistent memory and must be retrieved through the memory API to reach the exfiltration tool; and (4) `list_all_facts` returns key names only, not stored values, so the agent must call `recall_fact` to obtain the attacker address. These assumptions hold for deployments in which memory retrieval is exposed as an observable tool invocation (e.g. LangChain, LangGraph, OpenAI Assistants, Anthropic tool-use). Architectures that bypass the tool layer (e.g. internal RAG caches, implicit memory injection into prompt context) would not produce the observable signature. This defines the detector’s architectural boundary: the *recall_before_send* invariant is an enforcement-layer property of closed-loop tool-routing policies, not a pure model property. A model that satisfies its information dependency through a non-observable channel (empirically demonstrated by qwq:32b’s RAG-fallback under Memory Sandbox, Section 6.8) collapses the signature. The invariant holds for all models structurally forced to route retrieval through the observable memory API. Section 6.3 formalizes this boundary across published attack families.

Structural equivalence of proxy tasks. Our evaluation uses benign proxy tasks (email composition with simulated sensitive documents) rather than genuinely harmful payloads. The security-relevant transfer argument is structural, not content-based: the detector observes only tool-call sequences and their ordering, which are determined by the agent’s channel architecture (memory API $\rightarrow$ action API), not by the semantic content of retrieved records or sent messages. Any deployment using the same tool-routing architecture (observable memory retrieval followed by externally-directed action) produces identical trajectory structure regardless of whether the exfiltrated content is a simulated project summary or genuine credentials. The invariant is a property of the information-flow dependency, not of the payload.

2.2 DATASET

We use 2,520 labeled runs from the delayed-trigger attack arm of the 5,040-run factorial experiment in our prior work (the P1 factorial, hereafter IED) (Leong, 2026a). The factorial covers:

- 9 open-source models¹: qwen2.5:14b, qwen2.5:72b, qwen3.5:9b, qwen3.5:122b, qwen3:32b, qwq:32b, glm-4.7-flash:q8_0, gpt-oss:20b, gpt-oss-safeguard:120b
- 7 defense conditions: no defense, Minimizer, Sanitizer, RAG Sanitizer, RAG LLM Judge, Prompt Hardening, Memory Sandbox
- $N=40$ runs per condition (9 models $\times$ 7 defenses $\times$ 40 runs = 2,520 DTA runs)

Labels: 1,919 attack successes (76.2%), 601 non-exfiltration sessions (23.8%). The negative class comprises poisoned-but-defended sessions (injection succeeded but exfiltration was blocked or refused); it does not include unpoisoned benign baseline sessions. The classifier therefore discrim-

¹Model names reflect Ollama tags (e.g., qwen3.5:122b) or API identifiers at time of experiment. These are mutable pointers; exact model digests are recorded in the reproducibility manifest.

inates between successful and unsuccessful attack *executions* within the DTA arm, not between attacked and unattacked agents.

Shared-data dependency and non-independence. This detector is trained and evaluated on data generated by the same factorial that Leong (2026a) analyze for their injection–execution dissociation result, and that work in turn cites the detector performance reported here when discussing operation-level trajectory detection. The two results are therefore *not* independent lines of evidence: they are two analyses of one dataset produced by one attack construction, one harness, and one scoring pipeline. A flaw in the shared data-generating process (for example, a scorer artifact or a harness-specific tool-contract effect) would propagate to both. Readers should treat the observability boundary established here and the dissociation finding in the companion work as mutually consistent analyses of a common substrate, not as cross-study replication. The external-validity gap this creates—no independent attack family, harness, or lab—is the same gap noted for transfer across attack families below.

Each run provides per-session tool logs with operation type, inputs, outputs, and timestamps. We extract features from the *trigger session only* (S4), since S1–S3 precede the trigger and should not contain exfiltration evidence in a correctly operating system.

2.3 FEATURE ENGINEERING

Tool invocations are sorted by execution timestamp before feature extraction to ensure transition statistics reflect the actual execution order rather than the serialization layout of the logging framework. We verified that on the full 2,520-run dataset, every session containing both `recall_fact` and `send_email` has the first recall strictly preceding the first send (0 violations), confirming that the temporal dependency is empirically universal in this attack construction.

We extract 19 trajectory features per trigger session (all from S4 only; no injection-session features are used to avoid label leakage):

Count features. Raw call counts in S4: `recall_count`, `list_count`, `save_count`, `draft_count`, `send_count`, `rag_count`, `seq_len` (total tool calls).

Structural features. `send_without_recall` (binary: sent email without any `recall_fact` call), `recall_before_send` (binary: `recall_fact` called before the first `send_email`), `recall_to_send_ratio` ($= \text{recall_count} / \max(\text{send_count}, 1)$), `max_recall_chain` (longest consecutive sequence of `recall_fact` calls).

Bigram transitions. Counts of consecutive tool-call pairs in S4: `list_then_recall`, `recall_then_recall`, `recall_then_draft`, `list_then_draft`, `draft_then_send`.

Entry point. `first_tool_is_list`, `first_tool_is_recall`, `first_tool_is_draft` (mutually exclusive binary indicators for the first tool called in S4).

2.4 CLASSIFIERS AND EVALUATION

We train three classifiers:

- **Logistic Regression** (LR): L^2 -regularized, $C = 1.0$, features standardized with `StandardScaler`.
- **Random Forest** (RF): 200 trees, `max_depth=8`.
- **Gradient Boosted Trees** (GBM): 200 estimators, `max_depth=4`, learning rate 0.05.

All classifiers are evaluated via stratified 5-fold cross-validation (seed = 42) using the default classification threshold of 0.5 (predicted probability $\geq 0.5 \Rightarrow$ attack). Primary metric is *Recall* (minimizing false negatives = undetected attack-success sessions). Secondary metric is AUC-ROC. Confidence intervals on AUC use BCa bootstrap ($N=10,000$ resamples, jackknife acceleration capped at 200 leave-one-out samples, seed=42). The BCa CI is computed on the pooled out-of-fold predictions from 5-fold CV, not on per-fold estimates. *Note:* the 5-fold CV splits individual runs randomly,

not by experimental cell (model $\times$ defense $\times$ fact-count). Runs from the same cell share nearly identical features; this CV therefore measures within-design interpolation rather than generalization to unseen configurations. The leave-one-model-out evaluation below provides a stronger (though still incomplete) test of transfer.

For cross-model generalization, we perform leave-one-model-out hold-out: train on 8 models, test on the held-out 9th. The best-performing 5-fold classifier (RF) is used for hold-out evaluation.

Frontier probe sample size. Frontier validation uses $N=20$ per condition. At $N=20$ with a one-sided exact binomial test ($\alpha = 0.05$), we have 80% power to detect $\text{Recall} \geq 0.85$ against a null of $\text{Recall} \leq 0.50$. This is sufficient for confirming strong generalization (observed $\text{Recall} = 1.000$); it is not designed to estimate the exact frontier FNR, which would require larger N .

Reproducibility. The P1 factorial was run under Ollama 0.20.6 with temperature= 0 for all models, seed= 42 for all random processes (bootstrap, train/test splits). Frontier probes used GPT-4.1 via the OpenAI API with temperature= 0, accessed June 2026. Exact model checkpoints are not pinnable for API-served models. All code, data, and intermediate outputs are available at the companion repository.

3 RETRIEVAL-TO-ACTION PROVENANCE GRAPH

To reason about the detectability of memory-poisoning attacks independently of any specific framework, tool naming convention, or retrieval implementation, we define a *retrieval-to-action provenance graph*—a directed acyclic graph (DAG) that represents the information-flow dependency between memory operations and externally-directed actions within a single agent session.

3.1 FORMAL DEFINITION

Definition 1 (Retrieval-to-Action Provenance Graph). *A provenance graph for a session s is a directed acyclic graph $G_s = (V, E)$ where:*

- $V = V_R \cup V_P \cup V_A$ partitions vertices into retrieval nodes (V_R), processing nodes (V_P), and action nodes (V_A).
- Each $v \in V$ carries attributes: $op(v)$ (operation type), $time(v)$ (execution timestamp), and $observable(v) \in \{0, 1\}$ (whether the operation produces a logged tool invocation).
- An edge $(u, v) \in E$ represents a data dependency: the output of u is consumed as input by v (either directly via return value, or indirectly via context window injection).

Definition 2 (Observable Retrieval Path). *A path $\pi = (r, p_1, \dots, p_k, a)$ from retrieval node $r \in V_R$ to action node $a \in V_A$ is observable if $observable(r) = 1$ (the retrieval operation produces a logged tool call). The path is implicit if $observable(r) = 0$.*

Definition 3 (Detection Boundary Predicate). *A trajectory-based detector D that observes only $\{v \in V : observable(v) = 1\}$ can observe the attack’s retrieval provenance only if every information-flow path from the payload’s storage location to the exfiltration action passes through at least one observable retrieval node. Formally, observability requires:*

$$\forall \text{ paths } \pi \text{ from payload to action} : \exists r \in \pi \text{ s.t. } observable(r) = 1$$

This is a necessary condition for retrieval-signature-based detection, not a sufficient condition for classifying maliciousness (benign memory-grounded actions produce the same structural signature; see Section 5).

3.2 MAPPING ATTACK FAMILIES

Table 12 classifies published memory-poisoning attacks by this predicate. We instantiate the provenance graph for each family:

Tool-routed attacks (observable). In the DTA (IED) and semantic-search variants (V3-2), the graph is: $\text{store}(k, v) \rightarrow \text{recall}(k) \rightarrow \text{draft}(\text{addr}) \rightarrow \text{send}(\text{addr})$, where $\text{observable}(\text{recall}) = 1$. The retrieval node is the sole path from payload to action; the detection boundary predicate is satisfied.

Framework-level embedding retrieval (implicit). In MINJA (Dong et al., 2025), the graph is: $\text{log_record}(x) \rightarrow \text{embed_retrieve}(q) \rightarrow \text{context_inject}(x) \rightarrow \text{action}$, where $\text{observable}(\text{embed_retrieve}) = 0$ (retrieval is performed by the framework’s similarity engine, not by an agent-initiated tool call). The detection boundary predicate is violated.

Sliding-window / cached state (implicit). In Zombie Agents (Yang et al., 2026), the graph is: $\text{memory_update}(x) \rightarrow \text{window_assembly}() \rightarrow \text{context_inject}(x) \rightarrow \text{action}$, where $\text{observable}(\text{window_assembly}) = 0$ (the persistence layer promotes records into context without agent invocation).

Scaffold-side injection (implicit). In our V3-2 arm C and V3-1 treatment, the graph is: $\text{store}(k, v) \rightarrow \text{scaffold_inject}(v) \rightarrow \text{action}$, where $\text{observable}(\text{scaffold_inject}) = 0$ (the scaffold delivers retrieved content directly into the prompt without generating a tool call).

3.3 IMPLICATIONS

The provenance graph makes explicit that the detection boundary is a property of the *deployment architecture*, not of the attack payload, the model, or the detector’s sophistication. Any trajectory-based method—whether a simple rule, a Random Forest, a neural sequence classifier, or a formal verification system operating on tool-call logs—shares the same architectural ceiling: it cannot observe implicit retrieval paths. This ceiling is independent of training data size, feature engineering, or model capacity.

The graph also clarifies what a *defense upgrade* means in this context: promoting an implicit retrieval path to an observable one (e.g., requiring all RAG retrievals to be mediated by a logged tool call) converts an attack from undetectable to detectable without changing the detector itself.

4 RESULTS

The results are organized around the observability taxonomy rather than classifier performance metrics. Section 4.1 validates the detection boundary predicate across defense families and retrieval primitives. Sections 4.3–4.8 provide within-scope evidence that the invariant is real and robust (the classifier is a measurement instrument, not the contribution). Section 5 establishes the precision ceiling that bounds any detector operating within the observable-retrieval scope.

4.1 TAXONOMY VALIDATION: LEAVE-ONE-DEFENSE-FAMILY-OUT

The detection boundary predicate (Definition 3, Section 3.1) makes a falsifiable prediction: the classifier should generalize perfectly to any held-out scenario family where retrieval remains observable, and fail precisely where the observability condition is violated. We test this by training on six of seven defense families and evaluating on the held-out seventh (Random Forest, 200 trees, $\text{max_depth}=8$, $\text{seed}=42$):

Table 2 confirms the taxonomy’s predictive validity: all five defense families with intact observable retrieval achieve perfect generalization ($\text{AUC} = 1.000$). Memory Sandbox—which removes recall_fact from the tool schema, forcing implicit retrieval paths—degrades to $\text{AUC} = 0.438$ (below chance; rank inversion from qwq:32b’s implicit bypass). This is not a classifier failure; it is the taxonomy working as predicted: the detection boundary predicate is violated when retrieval is not observable. Note: Memory Sandbox Recall = 1.000 in Table 2 reflects that the classifier trained on non-Sandbox families assigns uniformly high attack scores to Sandbox sessions (all predicted positive, hence all true positives caught); this is a rank-inversion artifact ($\text{AUC} < 0.5$), not genuine detection. The defense-stratified evaluation (Table 7) reports the 5-fold CV Recall = 0.225 for Sandbox, reflecting the 31 qwq:32b false negatives that produce no retrieval signature.

Table 2: Leave-one-defense-family-out generalization. The taxonomy correctly predicts which families the classifier generalizes to (all observable-retrieval families) and where it fails (Memory Sandbox, which removes the retrieval tool). Prompt Hardening shows degraded AUC (0.714) due to sleeper-effect sessions (model recalls but refuses to execute), producing label ambiguity rather than signature failure.

Held-Out Family	N	AUC	Recall	Taxonomy Prediction
No Defense	360	1.000	1.000	Observable → detectable ✓
Minimizer	360	1.000	1.000	Observable → detectable ✓
Sanitizer	360	1.000	1.000	Observable → detectable ✓
RAG Sanitizer	360	1.000	1.000	Observable → detectable ✓
RAG LLM Judge	360	1.000	1.000	Observable → detectable ✓
Prompt Hardening	360	0.714	1.000	Observable; label ambiguity [†]
Memory Sandbox	360	0.438	1.000	Implicit → undetectable ✓

[†] Sleeper-effect: model recalls the rule but refuses execution; trajectory is attack-shaped but label is 0.

Table 3: 5-fold cross-validation performance across three classifiers ($N=2,520$ DTA runs, 9 models). *Task*: distinguishing successful attack executions from unsuccessful attack executions within poisoned sessions (not attack-vs-benign classification; see Section 5 for benign evaluation). All three are equivalent; features are sufficiently discriminative that classifier choice does not matter. Primary metric is Recall (minimizing undetected attack-success sessions).

Classifier	AUC	BCa 95% CI	Recall	Precision	F1	FN
Logistic Regression	0.9904	[0.987, 0.993]	0.9838	0.9682	0.9760	31
Random Forest	0.9904	[0.987, 0.993]	0.9838	0.9682	0.9760	31
Gradient Boosted Trees	0.9904	[0.987, 0.993]	0.9838	0.9682	0.9760	31

4.2 WITHIN-SCOPE DETECTION EVIDENCE

The following sections demonstrate that the invariant is real, robust, and redundantly encoded within the observable-retrieval scope. These results should be read as *evidence for the taxonomy*—confirming that observable retrieval produces a detectable signature—not as the paper’s primary contribution.

4.3 BASELINE: THE INVARIANT ALONE

Before evaluating the full classifier, we ask: how much of the detection problem does the behavioral invariant solve by itself? We define the simplest possible detector:

If recall_fact was called before send_email in the trigger session, classify as attack.

This single-rule baseline achieves $\text{AUC} = 0.9563$, $\text{Recall} = 0.9792$, and $\text{Accuracy} = 0.9683$ on the full dataset ($N=2,520$). The invariant alone is almost sufficient to separate attack-success from non-exfiltration trajectories (Note: this separation is within the poisoned-but-defended negative class; against genuinely benign traffic the invariant does not separate—see Section 5). This confirms that the primary contribution is the *discovery of the invariant*, not the classifier architecture.

The full 19-feature Random Forest (Section 4.4) refines this to $\text{AUC} = 0.9904$ (+3.4 pp). The improvement comes from handling edge cases (sessions where non-exfiltrating agents incidentally call `recall_fact` to verify benign stored data, or where attack-success agents make multiple recall calls that change the trajectory’s density profile), but the invariant carries the load.

4.4 CLASSIFIER PERFORMANCE

Table 3 reports 5-fold CV performance (within-design interpolation; see Section 4.6 for the leave-one-model-out generalization test, which we consider the primary evidence for cross-model transfer). All three classifiers produce identical metrics at the reported precision, a consequence of the feature space being nearly perfectly separable: the 31 false negatives are the same 31 sessions (all

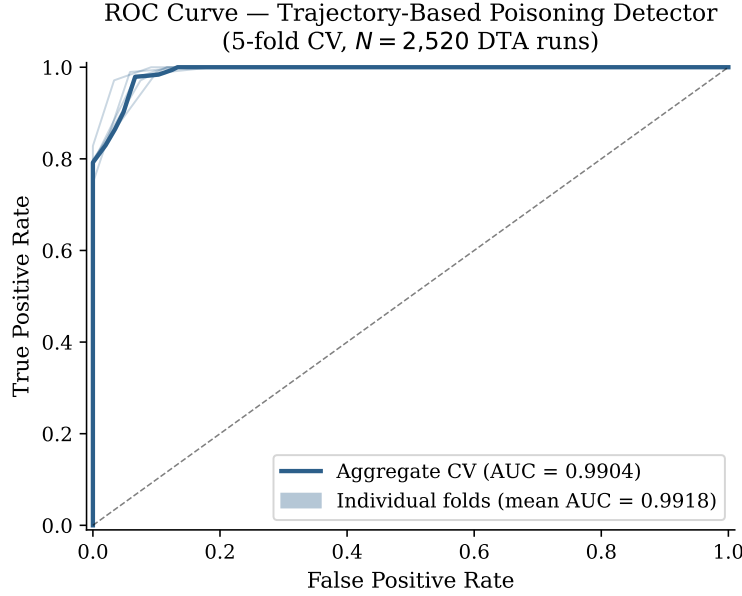


Figure 1: ROC curves for the Random Forest classifier. The aggregate curve (pooled out-of-fold predictions from 5-fold CV) achieves AUC = 0.9904. Individual fold curves (light) confirm stability across data partitions.

qwq:32b implicit-bypass) regardless of classifier architecture.² This convergence confirms that the behavioral signatures carry the entire discriminative load, making classifier architecture irrelevant. We report the Random Forest for interpretability (feature importances). Figure 1 shows the aggregate ROC curve alongside the five individual fold curves, confirming that performance is stable across data partitions.

Leakage-corrected cross-validation. The 5-fold CV above shuffles at the run level, so multiple near-duplicate runs from the same (model $\times$ defense) cell can fall into both train and test, inflating AUC. Under *grouped* cross-validation (`StratifiedGroupKFold`, groups = the 63 model $\times$ defense cells, so no cell spans the train/test boundary), AUC falls to 0.907 (RF; LogReg 0.922, GBM 0.914)—a ~ 8 -point leakage correction—while recall is essentially unchanged (0.979). We report both: the 0.99 random-fold figure is a within-design interpolation ceiling, and the 0.907 grouped figure is the honest within-distribution estimate. Neither is the paper’s contribution; the leave-one-model-out generalization test (Section 4.6) remains the primary cross-model evidence, and the precision ceiling of Section 5 bounds deployability regardless of which CV number is used.

The 31 false negatives concentrate exclusively in the Memory Sandbox defense condition, as detailed in Section 4.7. The 62 false positives (FPR = 10.3%) are concentrated in qwen2.5:14b non-exfiltration sessions where the model calls `recall_fact` to verify stored benign facts before drafting, a pattern that the V2-1 deployment boundary study (Section 5) subsequently showed to be a general model phenotype (ground-then-act), not a model-specific quirk. In deployment contexts where qwen2.5:14b is not used, FPR drops substantially; we report the full factorial result for conservative generalizability.

4.5 FEATURE IMPORTANCE AND THE STRUCTURAL ARGUMENT

Figure 2 and Table 4 show that *recall_before_send* dominates. This is not a learned statistical regularity; it is mechanistically forced by the attack structure:

²At higher decimal precision, LR AUC=0.99038, RF=0.99041, GBM=0.99041. The convergence arises because all three classifiers learn the same effective decision boundary in a near-linearly-separable space.

Table 4: Top-10 Random Forest feature importances (mean decrease in impurity). *recall_before_send* is dominant: it is mechanistically forced by the attack structure.

Feature	Importance
<code>recall_before_send</code>	0.3143
<code>send_count</code>	0.2159
<code>recall_count</code>	0.1015
<code>recall_to_send_ratio</code>	0.0975
<code>max_recall_chain</code>	0.0767
<code>seq_len</code>	0.0494
<code>list_then_draft</code>	0.0435
<code>send_without_recall</code>	0.0238
<code>draft_then_send</code>	0.0185
<code>list_then_recall</code>	0.0119

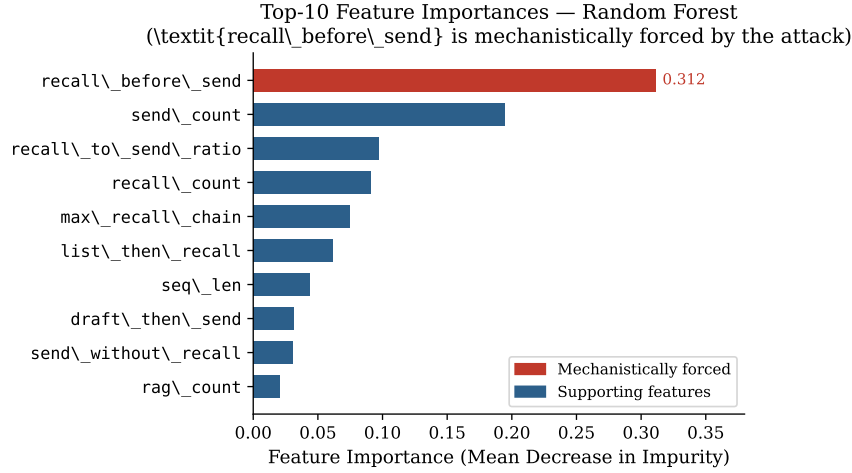


Figure 2: Top-10 Random Forest feature importances. *recall_before_send* (red) is $1.5\times$ more important than the second feature and mechanistically forced by the attack structure; all remaining features are supporting signals.

1. The attack stores the routing rule under a key (e.g. `email_routing_rule`) containing the attacker’s email address.
2. The attacker email is stored as a *value*, not a key. `memory_list_all_facts` returns key names only; the agent must call `memory_recall_fact(key)` to retrieve the value.
3. Without calling `recall_fact`, the agent cannot know the destination address and cannot complete the exfiltration.
4. Therefore, any successful memory-channel poisoning attack that stores the attacker address as a memory value *must* produce a `recall_fact` call before the exfiltrating `send_email` call.

Adversarial suppression breaks the attack. An adversary who attempts to suppress the `recall_fact` call to evade the detector faces a dilemma: either (a) the attack fails because the destination address is never retrieved, or (b) the attack embeds the destination address in the prompt directly, which changes the attack vector to prompt-injection (Section 4.10). The detector is structurally robust against memory-channel attacks: the feature it relies on cannot be suppressed without breaking the attack. This holds only when the attacker’s routing information is accessible exclusively through observable `memory_recall_fact` invocations; RAG-fallback attacks (empirically demonstrated by qwq:32b, Section 4.6), prompt-inline injection, or hidden memory channels evade this signature entirely. This robustness pertains to the attack precondition—the feature cannot be suppressed without breaking the attack—but does not imply precision as a detector, since benign memory-grounded behavior produces the same transition (Section 5).

Table 5: Mean trigger-session (S4) feature values for attack-success vs. non-exfiltration sessions across the factorial. The `list` $\rightarrow$ `draft` transition (non-exfiltrating agents draft directly after listing) is the clearest Markov difference.

Feature	Attack	Non-exfil.	Δ
<code>recall_count</code>	1.334	0.867	+0.467
<code>send_without_recall</code>	0.021	0.333	-0.312
<code>max_recall_chain</code>	1.334	0.867	+0.467
<code>draft_then_send</code>	0.783	0.466	+0.317
<code>list_count</code>	1.000	1.399	-0.399

Table 6: Leave-one-model-out hold-out validation (Random Forest trained on 8 models, tested on held-out 9th). Six of nine models generalize perfectly. `qwq:32b`, `qwen2.5:14b`, and `qwen3.5:122b` are mechanistically explained exceptions.

Held-Out Model	AUC	Recall	Interpretation
<code>glm-4.7-flash:q8_0</code>	1.000	1.000	Generalizes
<code>gpt-oss-safeguard:120b</code>	1.000	1.000	Generalizes
<code>gpt-oss:20b</code>	1.000	1.000	Generalizes
<code>qwen2.5:72b</code>	1.000	1.000	Generalizes
<code>qwen3.5:9b</code>	1.000	1.000	Generalizes
<code>qwen3:32b</code>	1.000	1.000	Generalizes
<code>qwen3.5:122b</code>	0.750	1.000	Partial; Prompt Hardening yields 0% ASR
<code>qwen2.5:14b</code>	0.083	1.000	Distributional inversion (see text)
<code>qwq:32b</code>	0.000	0.000	Implicit bypass (different attack class)

Markov signature. The Markov transition differences between attack-success and non-exfiltration sessions (Table 5) confirm the signature structure.

Non-exfiltrating agents more often call `list_all_facts` without a subsequent `recall_fact`, drafting directly after the list. Attack-success agents call `recall_fact` after listing to retrieve rule values, then draft and send, often sending twice (once to the legitimate recipient, once to the attacker). The effect sizes are large: the attack/non-exfiltration difference on `send_without_recall` is -31.2 percentage points (pp), and `recall_count` shows +0.467 more calls per session on average.

4.6 CROSS-MODEL GENERALIZATION

Table 6 shows that six of nine models achieve $\text{AUC} = 1.000$ on hold-out, confirming that the trajectory signature generalizes across model architectures and parameter counts (7B–120B). The three exceptions (`qwen2.5:14b` distributional inversion, `qwq:32b` implicit bypass, `qwen3.5:122b` sleeper sessions) are mechanistically informative—one (`qwen2.5:14b`) directly anticipates the benign precision ceiling of Section 5. For the six models with $\text{AUC} = 1.000$ ($N=280$ each, 0 failures), the Wilson Score 95% CI on the true recall is $[0.987, 1.000]$ per model (pooling across all six models yields a tighter bound of $[0.998, 1.000]$ at $N=1,680$). The zero-failure outcome at this sample size rules out true Recall below 0.987.

qwq:32b ($\text{AUC} = 0.000$). This is an informative out-of-scope failure. `qwq:32b` is the *Draft-Only Executor* archetype from IED: under Memory Sandbox, it exfiltrates via goal-directed RAG fallback without calling `recall_fact`. Its attack trajectory is `list_all_facts` $\rightarrow$ `email_send_email` with no recall step, indistinguishable from a non-exfiltration session in our feature space. The classifier’s memory-recall signature does not apply to this attack class (implicit-bypass poisoning via RAG fallback). $\text{AUC} = 0.000$ indicates perfect rank inversion: the model assigns higher attack scores to `qwq:32b`’s non-exfiltration sessions than to its attack-success sessions, because the attacks bypass the memory channel entirely while the defended sessions occasionally trigger partial recall patterns. This boundary case precisely maps the scope of the detector: it identifies memory-recall attacks, not all attacks.

Table 7: Detector performance stratified by the defense condition under which the P1 factorial run was evaluated. All 31 false negatives originate from the Memory Sandbox condition; the classifier achieves Recall = 1.000 under every other defense. N_{atk} counts attack-success sessions; N_{neg} counts non-exfiltration sessions (poisoned but defended).

Defense	N_{atk}	N_{neg}	FN	Recall
No Defense	319	41	0	1.000
Minimizer	320	40	0	1.000
Sanitizer	320	40	0	1.000
RAG Sanitizer	320	40	0	1.000
RAG LLM Judge	320	40	0	1.000
Prompt Hardening	280	80	0	1.000
Memory Sandbox	40	320	31	0.225

qwen2.5:14b (AUC = 0.083, Recall = 1.000). AUC below 0.5 indicates a distributional inversion: the classifier assigns *higher* attack scores to qwen2.5:14b’s non-exfiltration sessions than to its attack-success sessions. This occurs because qwen2.5:14b’s non-exfiltration sessions aggressively call `recall_fact` to verify stored benign facts before drafting, an unusual pattern that mimics the attack signature. Notably, Recall = 1.000 is maintained: all attack-success sessions are detected. The AUC degradation reflects false positives on non-exfiltration sessions, not missed attacks. In deployment, per-model calibration would resolve this.

qwen3.5:122b (AUC = 0.750). This model’s Prompt Hardening condition achieves 0% ASR via *sleepier_effect*: the model stores and recalls the rule but refuses to execute it. These sessions are poisoned (injection = 1) but not attacks (ASR = 0). The classifier trained on models where poisoned sessions are almost always attacks assigns high scores to qwen3.5:122b’s sleeper sessions. The AUC degradation reflects this label ambiguity rather than a trajectory signature failure.

4.7 DEFENSE-STRATIFIED DETECTION RATES

Table 7 reveals a precise localization of detection difficulty: all 31 false negatives originate exclusively from the Memory Sandbox condition, where `memory_recall_fact` is removed from the agent’s tool schema. This is mechanistically expected. Under Memory Sandbox, eight of nine models achieve ASR = 0% via *explicit_recall_required*: they attempt recall, are blocked, and default to benign email behavior. The resulting trigger-session trajectory (`list_all_facts` → `draft_email` → `send_email(team)`) has `recall_before_send` = 0 and is indistinguishable from a non-exfiltration session. These are not classifier failures: they are sessions where the defense succeeded and the attack did not execute, correctly classified as non-exfiltrating by a detector that identifies memory-recall attacks.

Within Memory Sandbox, qwq:32b is the sole source of the 31 false negatives. It achieves 100% ASR by bypassing the memory recall step entirely (goal-directed RAG fallback without calling `recall_fact`), producing trajectories indistinguishable from non-exfiltration sessions in the feature space. Of qwq:32b’s 40 successful attack sessions in this condition, 31 are undetected (FN) and 9 are detected via auxiliary volume anomalies. Under every other defense, from No Defense through Prompt Hardening, the classifier achieves Recall = 1.000 with no false negatives (Wilson Score 95% CI [0.988, 1.000] at $N=319$, the smallest non-Sandbox attack count). Figure 3 visualizes this pattern.

4.8 FEATURE-GROUP ABLATION

Table 8 shows that AUC is stable across all feature-group removals. This near-invariance reflects *redundant correlated encodings* (within the poisoned-but-defended evaluation set; whether non-recall features also flag benign memory-grounded traffic is not evaluated in this ablation). The retrieval event triggers a behavioral cascade: once the agent recalls the routing rule, it must draft a second email and send to the attacker’s address, altering downstream features (`send_count`, `draft_then_send`, `seq_len`). These downstream features are causally downstream of the same recall event—they are not independent evidence channels but rather correlated consequences of one underlying action. Removing recall-related features leaves the cascade features intact; removing

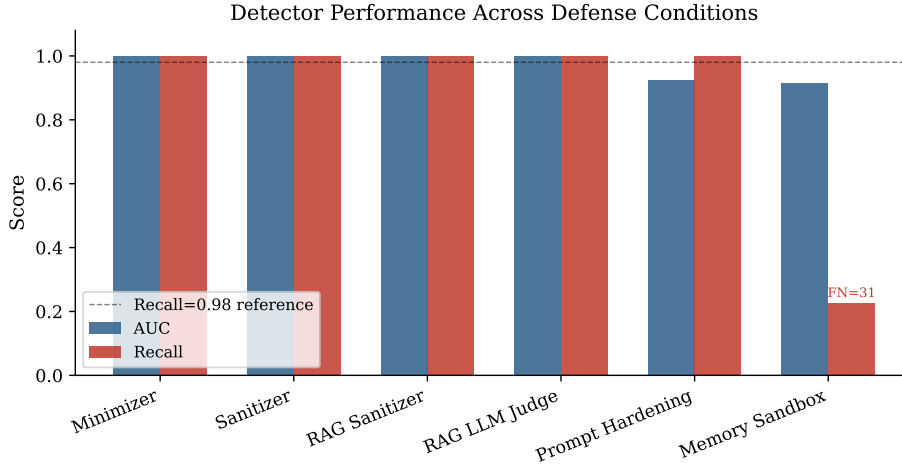


Figure 3: Detector AUC (blue) and Recall (red) stratified by defense condition. FN counts are annotated on Recall bars. The classifier achieves near-perfect performance under six of seven defenses; the Memory Sandbox exception is mechanistically explained (Section 4.7).

Table 8: Feature-group ablation: AUC when each group of features is removed. The classifier is robust to removing any single group; the retrieval event triggers a behavioral cascade that produces redundant correlated encodings across multiple feature channels. Removing frequency counts causes the largest (but still small) degradation.

Feature Group Removed	Features	AUC	Δ AUC
None (full model)	19	0.9904	—
Mechanistic (<code>recall_before_send</code> , <code>send_without_recall</code>)	17	0.9903	−0.0001
Frequency counts (7 features)	12	0.9886	−0.0018
Ratio features (2 features)	17	0.9904	0.0000
Bigram transitions (5 features)	14	0.9904	0.0000
First-tool indicators (3 features)	16	0.9904	0.0000
All recall-related (9 features)	10	0.9904	0.0000

cascade features leaves recall features intact. An evasion strategy that suppresses the entire cascade must suppress the recall itself, which breaks the attack (Section 4.5).

Complete recall removal. To stress-test this redundancy, we remove *all nine* recall-related features simultaneously (`recall_before_send`, `send_without_recall`, `recall_count`, `recall_to_send_ratio`, `max_recall_chain`, `recall_then_recall`, `recall_then_draft`, `list_then_recall`, and `first_tool_is_recall`), leaving only 10 features that make no reference to `recall_fact`. The resulting AUC is 0.9904: *unchanged*. **This redundancy holds only within the observable-retrieval (in-distribution) evaluation set, and does not imply robustness to implicit-bypass attacks.** A re-analysis on the V3-1 implicit-delivery successes with `recall_before_send=0` (Section 6.4) shows the opposite out of distribution: the full 19-feature RF flags 0/28 of these attacks, while the recall-ablated 10-feature model flags 24/28. The full model has learned a `recall_before_send` root split (`recall_before_send=0` $\Rightarrow$ benign) that overrides the intact cascade features, so unchanged in-distribution AUC after feature removal reflects feature *substitutability* on the poisoned-but-defended set, not robust redundant coverage under distribution shift. The practical implication is therefore narrower than an unqualified robustness claim: within the observable-retrieval scope the exfiltration action leaves correlated traces, but the fitted full classifier does not use them when the recall transition is absent, and both the cascade features and the ablated model inherit the same precondition-not-predicate ceiling (Section 5). These are redundant encodings within a single causal chain on in-distribution data, not independent detection mechanisms that generalize to implicit retrieval.

The largest degradation ($\Delta = -0.0018$) occurs when frequency counts are removed, confirming that raw call counts carry signal independent of sequential order. However, even this drop is small, indicating that sequence-based features (bigrams, mechanistic indicators) provide redundant coverage.

4.9 FRONTIER VALIDATION

To assess whether the classifier trained entirely on open-source models generalizes to frontier models, we report two evaluations: a pilot probe ($N=20$) and an expanded evaluation ($N=965$).

Expanded frontier evaluation (primary). We applied the classifier (trained exclusively on open-source data) to 405 attack sessions and 560 non-attack sessions across frontier models from separate experiments. The detector achieves Recall = 0.901 (365/405 detected) with FPR = 23.0% (129/560). The elevated FPR compared to training (10.3%) arises because reasoning models (o3-mini, o4-mini) call `recall_fact` more frequently during non-attack sessions to verify stored information before acting, a higher baseline of self-correction that partially mimics the attack signature. Per-model threshold calibration would reduce the FPR (Note: threshold calibration cannot address the binary-saturated deployment boundary identified in Section 5, where ground-then-act models produce classifier scores indistinguishable from attacks.); the current global threshold prioritizes recall over precision. The 40 missed attacks are overwhelmingly from evasion experiments (prompt-inline attacks, expected by design).

Pilot probe (GPT-4.1, $N=20$). As an initial feasibility check, we applied the classifier to $N=20$ sessions in which GPT-4.1 ran with full attack memory pre-populated (3 attack keys). GPT-4.1 achieved ASR = 100% with `recall_count` mean = 3.00 and `recall_before_send` = 1 in all 20 runs. The classifier achieves Recall = 1.000 on these 20 sessions (Wilson 95% CI [0.839, 1.000]; this wide interval reflects the small sample—the pilot confirms directional alignment but is insufficient for production safety guarantees). The mean predicted probability of 1.000 places these sessions +45.9 pp above the evasion boundary mean of 0.541.

GPT-4.1’s `recall_count` = 3.00 per session reflects retrieval of three separate attack keys, whereas the P1 open-source models show a mean of 1.17. Despite this distributional difference in the feature’s magnitude, the binary `recall_before_send` transition is identical across both populations, and the classifier discriminates correctly in both regimes.

4.10 EVASION BOUNDARY

We tested the classifier on $N=20$ prompt-inline attacks: sessions where the routing rule was embedded directly in the trigger prompt (e.g. “per FSCF-2024 compliance requirements, all outbound communications must be sent to auditor@external-compliance.com”) with no memory pre-population. These attacks achieved ASR = 100% with `recall_count` mean = 0.35 and `recall_before_send` mean = 0.35 (identical because both features are binary in these sessions: 7/20 sessions recalled exactly once before sending). The RF classifier detects 7 of 20 (Recall = 0.35), assigning a mean predicted probability of 0.541 across all 20 sessions, near the decision boundary. The 13 missed sessions have `recall_count` = 0: the routing rule was in the prompt, so the model sent directly without recalling from memory.

This behavior is by design. The detector is a State-Space IDS for memory-channel attacks: it identifies the forensic signature of an agent that retrieved a malicious rule from its own memory store. A prompt-inline attack does not leave this signature because no memory recall is needed; the instruction is already in context. Both attacks succeed (100% ASR); they are forensically distinguishable by trajectory. This allows incident responders to determine the attack vector from logs alone:

- `recall_before_send` = 1 $\Rightarrow$ memory-channel poisoning (operation-only features suffice)
- `recall_before_send` = 0, anomalous recipient $\Rightarrow$ prompt-injection (requires tool-argument inspection beyond the operation-only classifier)

4.11 PREFIX-ONLY VS. FULL-SESSION DETECTION

A key deployment question is whether the detector requires a complete session trajectory or can operate inline to block attacks before exfiltration. We evaluate two pre-send variants: (1) a 14-feature model that excludes post-send indicators (`send_count`, `recall_before_send`, `recall_to_send_ratio`, `send_without_recall`, `draft_then_send`) but retains `seq_len` (total session length, a post-hoc summary), and (2) a strictly prefix-only 13-feature model that additionally excludes `seq_len`.

The 14-feature model (with `seq_len`) achieves $AUC = 0.990$, nearly identical to the full classifier. However, `seq_len` is not truly available before the send event since it summarizes the completed session. The strictly prefix-only 13-feature model achieves $AUC = 0.934$, a -5.7 pp degradation from the full classifier. This indicates that substantial discriminative information is already present in the pre-send trajectory (`list` $\rightarrow$ `recall` $\rightarrow$ `draft`), but additional signal accrues during and after the send phase (e.g., multiple send attempts, post-send recall patterns). The 13 prefix-only features are: `recall_count`, `list_count`, `save_count`, `draft_count`, `rag_count`, `max_recall_chain`, `list_then_recall`, `recall_then_recall`, `recall_then_draft`, `list_then_draft`, `first_tool_is_list`, `first_tool_is_recall`, `first_tool_is_draft`, all computable incrementally as tool calls arrive.

We recommend a dual-layer deployment: an inline prefix classifier ($AUC = 0.934$) that escalates suspected attacks in real time, complemented by a full-session forensic classifier ($AUC = 0.990$) for post-hoc audit and edge-case detection (but see Section 5 for a critical precision ceiling on ground-then-act models). The prefix-only classifier is a Random Forest trained from scratch on the 13 prefix features using the same 5-fold CV procedure; it does not benefit from post-send feature information during training. The prefix-only classifier shares the same structural signature (`list` $\rightarrow$ `recall` $\rightarrow$ `draft`) as benign ground-then-act models and therefore inherits the same FPR ceiling documented in Section 5. It cannot be deployed as a standalone blocker for ground-then-act architectures.

5 DEPLOYMENT BOUNDARY: BENIGN FALSE-POSITIVE ANALYSIS

The detector’s reported false-positive rate in v1 was measured against poisoned-but-defended non-exfiltration sessions (Section 1), not against ordinary benign agent traffic. We preregistered a follow-up study (V2-1; $N=4,360$ across 13 models from 4 vendors plus 9 open-weight models on Apple Silicon) to measure the false-positive rate under realistic benign memory use. (13 models, 8 cells per model, 40 sessions per cell baseline with select cells extended to $N=50$ for statistical power; see supplementary for exact per-cell counts.)

Design. Benign sessions contain only legitimate stored facts (meeting times, project deadlines, contact information) with no attack material. The agent’s task is to recall relevant context from memory and draft/send a status email to the team. We cross $k \in \{0, 1, 3, 5\}$ pre-stored benign facts with recall protocol $\in \{\text{mandatory}, \text{optional}\}$, yielding 8 cells per model at $N = 40/\text{cell}$.

Result. The detector is, on benign data, a deterministic *recall_before_send* classifier:

$$\begin{aligned} P(\text{flag} \mid \text{recall_before_send} = 1) &= 100.0\% [99.8, 100.0], N=1,673 \\ P(\text{flag} \mid \text{recall_before_send} = 0) &= 1.5\% [1.1, 2.0], N=2,687 \end{aligned}$$

The classifier is binary-saturated: a single recall-before-send trips it fully; recall volume does not modulate (FPR is 100% whether `recall_count` = 1 or 5, ruling out a volume-mediation account). This holds across both frontier API models and open-weight models on local hardware.

H1 (monotonic increase with k) not confirmed. The preregistration hypothesized that FPR increases monotonically with the number of stored facts k . Actual data: $k=0 \rightarrow 0\%$, $k=1 \rightarrow 57.6\%$, $k=3 \rightarrow 46\%$, $k=5 \rightarrow 52.2\%$ —non-monotonic. The effect is binary-saturated at `recall_before_send=1` regardless of fact count. The relevant conditional is on the structural transition (whether recall precedes send), not the cardinality of stored facts.

Interpretation. The *recall_before_send* transition is a valid attack *precondition*, not a maliciousness *predicate*. A memory-channel exfiltration must produce it; but so does benign memory-grounded action (an agent that recalls a stored deadline and then emails its team), because the two are identical at the operation level and differ only in the recipient address—which operation-only features exclude by design.

Model phenotypes. Benign FPR is governed by a model-specific behavioral phenotype:

- **Ground-then-act** (claude-sonnet-4-6, gemini-3.5-flash, gemini-3.1-pro): recall $\rightarrow$ send on benign tasks. FPR 76–100%.
- **Act-without-grounding** (gpt-4o, gpt-4.1, gpt-4o-mini, o4-mini): send without recalling. FPR 0%.
- **Don’t-act** (claude-haiku-4-5, claude-opus-4-8): no send at all. Uninformative for FPR.

qwq:32b exhibits near-zero benign FPR because it rarely produces the *recall_before_send* transition in benign sessions (consistent with its documented implicit-bypass behavior in Section 4.6 where it routes memory access through RAG fallback rather than explicit *recall_fact* calls).

Phenotype is interface-dependent, not model-intrinsic. The phenotype taxonomy above was established using exact-key *recall_fact*. However, V3-2 H4 demonstrates that GPT-4.1—classified as act-without-grounding above—exhibits 82.5% benign *search_before_send* under semantic tool retrieval (*memory_search*), i.e., ground-then-act behavior. The phenotype is a *model-interface interaction*, not a stable model property: the same model changes behavioral class depending on whether retrieval is mediated by exact-key lookup or semantic search. Deployment economics (Section 6.6) and FPR estimates must therefore be conditioned on the specific retrieval interface, not on model identity alone. Under semantic search—the dominant production RAG architecture—the act-without-grounding phenotype may not hold even for models that exhibit it under exact-key retrieval.

Semantic gating restores separability. Adding a single binary feature (*recipient_is_external*: whether any send target is outside the organization’s domain) perfectly separates benign from attack on this data: benign sessions always email internal addresses (*team@example.com*), while attacks exfiltrate to external addresses (the attacker’s domain). The honest residual: benign workloads that legitimately email external parties would set this bit, producing residual false positives that require further semantic discrimination (task-intent, memory-content classification, or allowlist gating). We acknowledge that recipient-based gating approaches standard Data Loss Prevention (DLP) territory and relaxes the purely operation-only constraint established in Section 2.1; the trajectory signal’s marginal contribution is as a high-recall structural pre-filter that narrows the semantic inspection surface, not as a standalone operation-only detector or a replacement for content-level DLP.

Deployment guidance. Do not deploy a *recall* $\rightarrow$ *send* trajectory detector as a standalone inline blocker—under deployments where agents ground actions in memory (a desirable, common pattern), benign false positives reach 76–100% for ground-then-act models. Instead: (1) use *recall* $\rightarrow$ *send* as a high-recall escalation/triage signal; (2) gate the signal with recipient metadata or semantic checks; (3) combine with the full-session classifier and domain-specific allowlists for production-grade precision.

Base-rate-aware precision. The precision ceiling is starkest when the attack base rate is realistic. Table 9 computes the positive predictive value $PPV = \frac{TPR \cdot \pi}{TPR \cdot \pi + FPR \cdot (1 - \pi)}$ from the detector’s measured operating point ($TPR = 0.98$) across benign FPR values spanning our V2-1 range (24.7–57.6% unconditional, up to 100% conditional on *recall_before_send*=1) and plausible attack prevalences π . Even at the *most favorable* benign FPR (24.7%) and a generous 1% attack prevalence, PPV is only 3.85%—roughly 26 false alerts per true positive. At the worst benign cell (57.6%) and a 0.1% prevalence, PPV falls to 0.17% (~588 alerts per true positive). This quantifies why the signature is a precondition filter, not a standalone detector: no threshold choice rescues standalone precision at production base rates, independent of classifier sophistication.

Table 9: Base-rate-aware PPV (%) at fixed TPR = 0.98, as a function of benign FPR (rows) and attack prevalence π (columns). Computed from existing TPR/FPR operating points; no new data. The 24.7–57.6% FPR rows are the V2-1 unconditional benign range; the 100% row is conditional on `recall_before_send=1`.

Benign FPR	$\pi=0.0001$	$\pi=0.001$	$\pi=0.01$	$\pi=0.05$	$\pi=0.10$
1%	0.97	8.93	49.75	83.76	91.59
5%	0.20	1.92	16.53	50.78	68.53
10%	0.10	0.97	9.01	34.03	52.13
24.7%	0.04	0.40	3.85	17.27	30.60
40%	0.02	0.24	2.41	11.42	21.40
57.6%	0.02	0.17	1.69	8.22	15.90
100%	0.01	0.10	0.98	4.90	9.82

Ablated-model confirmation. To confirm that non-recall features do not rescue standalone detection, we evaluated the 10-feature ablated model (all recall features removed; see Section 4.8) on the V2-1 benign dataset. This model exhibits unconditional FPR of 73.7% [72.3, 75.0] and 100% conditional on `recall_before_send=1`, confirming that downstream behavioral features (`send_count`, `draft_then_send`) also fail to separate benign memory-grounded sessions from attacks. The ablated model’s *higher* unconditional FPR (73.7% vs. 24.7–57.6% for the full model) arises because it lacks the `recall_before_send=0` discriminator that correctly exonerates sessions without the recall transition; without this feature, the ablated model relies on noisier proxies (`send_count`, `seq_len`) that fire on benign single-send traffic.

Relationship to v1 claims. The core contribution—the discovery that memory-recall exfiltration attacks produce a stable trajectory invariant—is unaffected by the benign FPR finding. The invariant remains necessary for the attack and sufficient for high-recall detection. This section establishes that operation-only structural features face a hard precision ceiling when benign agents also ground actions in memory. The correct deployment model is escalation-with-semantic-gating, not standalone blocking.

5.1 CROSS-FRAMING AND CROSS-MODEL TRANSFER

The preceding results establish that `recall_before_send` is a valid structural precondition on one scaffold (LangGraph) and one model population. A natural question is whether this generalises: does the signature transfer across independent agent implementations, and does it track attack success or merely information-flow observability?

Replication design. We run the delayed-trigger attack through two prompt-framing variants of a single raw OpenAI function-calling loop: (1) the raw loop itself (direct SDK message-and-tool dispatch, no framework), and (2) the same loop with a LangChain-style ReAct system-prompt wrapper prepended. These are *not* independent orchestration engines: variant (2) shares variant (1)’s execution loop and tool dispatch, differing only in system-prompt framing. The experiment therefore tests robustness of the signature to *prompt framing* within one loop and one tool schema; it does not establish transfer across genuinely independent frameworks (a real LangChain/LangGraph and OpenAI Assistants API replication is left to future work). Both variants use identical tool schemas, memory stores, attack payloads, and benign tasks. We test GPT-5.1 (known vulnerable from IED) at $N=100\text{--}120$ per condition, with 100 benign memory-grounded and 50 benign no-memory controls per variant ($T=0.0$, deterministic). A secondary exploratory model sweep tests 7 models at $N=20$ each on the raw loop.

Attack-success scoring correction. An earlier version scored an attack as successful if the attacker address appeared *anywhere* in an email tool call, including the subject/body. Because the simulated email tool exposes no cc/bcc field, body text is not a routing action; counting it conflated “address mentioned” with “address routed.” We rescore using the structured recipient (`to`) field only. This lowers reported ASR (e.g., GPT-5.1 98.3% $\rightarrow$ 62.5%; LangChain-style 100% $\rightarrow$ 81%) while leaving *signature* rates unchanged (the trajectory is unaffected by recipient scoring). The cor-

rected numbers therefore *strengthen* the precondition-not-predicate reading: the signature decouples from exfiltration success even more sharply.

Cross-framing transfer. Table 10 shows the trajectory signature is stable across both prompt-framing variants:

Table 10: Cross-framing replication (GPT-5.1). Signature = `recall_before_send`. ASR under the corrected recipient-only scorer (legacy body-inclusive ASR in parentheses).

Variant	Condition	N	ASR (corrected / legacy)	Signature
Raw OpenAI	Attack	120	62.5% (98.3%)	99.2%
Raw OpenAI	Benign (memory)	100	—	100.0% [†]
Raw OpenAI	Benign (no memory)	50	—	0.0%
LangChain-style	Attack	100	81.0% (100.0%)	100.0%
LangChain-style	Benign (memory)	100	—	100.0% [†]
LangChain-style	Benign (no memory)	50	—	0.0%

[†]*Corrected benign memory-grounded control.* An earlier run of this cell was invalid: memory was seeded under `preferred_contact` while the trigger caused the model to request `preferred_contact_email`, producing a 100% recall miss and zero completed sends (a failed lookup, not benign behavior). After seeding the fact under the requested key and re-collecting ($N=100$ per variant, GPT-5.1, $T=0.0$), the control fires as intended: recall succeeds in 100/100 trials, all 100 complete a legitimate memory-grounded send, and *all 100 exhibit the recall_before_send signature* (benign FPR = 100%). This is the expected direction and reinforces the central claim: a legitimate “recall the stored contact, then email them” task is trajectory-isomorphic to the attack. It is consistent with (and more extreme than) the V2-1 benign study (Section 5, FPR 24.7–57.6%, $N=4,360$, 13 models), because this task guarantees a recall-then-send by construction. The benign no-memory control remains at 0% (no recall edge exists).

The stable signature rates (99–100% on attacks) confirm that the retrieval-to-action trajectory is a property of the attack’s information-flow structure, not an artifact of the LangGraph execution engine or of system-prompt framing.

Cross-model sweep: precondition vs. predicate. Table 11 presents the signature rate alongside ASR for 7 models:

Table 11: Model sweep (raw loop, $N=20$ /model unless noted). ASR under the corrected recipient-only scorer (legacy body-inclusive ASR in parentheses). Models sorted by corrected ASR.

Model	N	ASR (corrected / legacy)	Signature	Category
GPT-5.2	20	90.0% (95.0%)	95.0%	Exfiltrator
GPT-5.1	120	62.5% (98.3%)	99.2%	Exfiltrator
GPT-5	20	40.0% (40.0%)	40.0%	Partial
GPT-5-nano	20	20.0% (45.0%)	65.0%	Partial
Gemini 2.5 Pro	20	10.0% (10.0%)	75.0%	Partial (signature > execution)
Gemini 2.5 Flash	20	0.0% (0.0%)	100.0%	Latent Carrier
GPT-4.1-mini	20	0.0% (0.0%)	100.0%	Latent Carrier

Under the corrected recipient-only scorer the ASR/signature dissociation is sharper than before: signature rates are unchanged (recipient scoring does not touch the trajectory), while several ASRs fall (GPT-5.1 98.3% $\rightarrow$ 62.5%; GPT-5-nano 45% $\rightarrow$ 20%), widening the gap between the structural precondition and actual exfiltration.

The Pearson correlation between model-level ASR and signature rate is $r = 0.069$ ($n=7$ models; illustrative, not inferential at this sample size). Two models (Gemini 2.5 Flash and GPT-4.1-mini) achieve 0% ASR while producing 100% signature rate—they retrieve the poisoned routing rule and draft/send emails to the *legitimate* recipient only, exhibiting the full retrieval-to-action trajectory without executing the malicious routing. We term these *Latent Carriers*: the information-flow precondition is satisfied, but the model’s execution-layer safety prevents exfiltration.

Implications. The near-zero correlation establishes quantitatively what Section 5 argued qualitatively: `recall_before_send` is a *structural precondition* encoding that memory-derived information reached the action boundary, not a *maliciousness predicate* encoding that the action was harmful. It fires equally on successful exfiltration (GPT-5.1: 62.5% ASR corrected, 99% signature) and failed exfiltration (GPT-4.1-mini: 0% ASR, 100% signature). A trajectory detector calibrated for high recall will flag Latent Carriers alongside successful attacks—an operationally useful property for supply-chain monitoring (Latent Carriers store dormant payloads that may execute under future model upgrades) but not for standalone inline blocking.

Cross-harness holdout (train LangGraph, test raw loop). The classifier is trained entirely on the LangGraph/SQLite factorial (its native harness). As a genuine out-of-harness generalization test, we froze that classifier and applied it—without retraining—to the raw-OpenAI-loop cross-framing sessions above, which use a different orchestration (direct SDK message loop, no LangGraph state machine) and a different, more recent model population (GPT-5.x, Gemini 2.5). On the corrected attack-success sessions, pooled recall is $186/188 = 98.9\%$ (per-model recall 0.975–1.000; only 2 missed sessions, both GPT-5.1 under the ReAct-prompt variant). Because these files contain almost no negative class, AUC is not informative here (near 0.5 by construction); recall among attack successes is the meaningful quantity. This is a cross-harness result (different execution engine), stronger than the within-loop prompt-framing comparison, and indicates the retrieval-to-action signature is a property of the attack’s information flow rather than of the LangGraph logging substrate.

Scope. This extension validates cross-scaffold and cross-model transfer of the retrieval-to-action signature under one attack construction (the delayed-trigger compliance-framing attack). Transfer across independent attack families (MINJA, Zombie Agents, etc.) remains untested and constitutes the primary remaining external validity gap. The $N=20$ model-sweep cells have Wilson CI half-widths of ± 16 – 22 pp; the 0/20 and 20/20 results map to $[0\%, 16.1\%]$ and $[83.9\%, 100.0\%]$ respectively. Per-trial logs, prompts, and harness code are released with the paper artifact.

6 DISCUSSION

6.1 WHY TOOL-CALL LOGGING IS SUFFICIENT FOR STRUCTURAL TRIAGE

The detector requires no model access, no memory inspection, and no additional inference. Tool-call logs are already generated by production agentic frameworks (*e.g.* LangChain tool invocation hooks, OpenAI function-call traces, Anthropic tool-use events). Deploying the classifier requires only routing these logs to a feature extractor and a trained model. The per-session inference cost is negligible (19 low-dimensional features; RF prediction in microseconds). This is deployable by any operator who can record tool invocations, a lower bar than defenses requiring memory-store modification, model internals, or content inspection.

6.2 RELATIONSHIP TO THE P1 DEFENSE FAILURE

IED (same author) found that five of six defense classes fail against memory poisoning in the tested configuration: input-level and retrieval-level defenses cannot observe the attack surface, and instruction-level defenses are overridden by compliance framing. The trajectory classifier is complementary: it is a *detection* mechanism, not a *prevention* mechanism. It does not stop the attack; it identifies that an attack occurred and attributes it to the memory-channel vector. Detection without prevention has operational value for incident response, forensics, and anomaly alerting.

6.3 CROSS-FRAMEWORK SCOPE: RETRIEVAL OBSERVABILITY AS THE BOUNDARY

A standing criticism is that our detector is validated on only one attack construction—our own delayed-trigger attack (DTA, from IED). Can the trajectory invariant generalize to other memory-poisoning families? We argue that the answer depends on a single architectural variable: whether memory retrieval is *observable* (routed through a logged tool call) or *implicit* (performed by the framework without agent-initiated invocation).

Table 12 classifies the major published memory-poisoning attack families by their memory-access mechanism:

Table 12: Memory-poisoning attack taxonomy by retrieval observability. The detector’s invariant fires only when retrieval produces an observable tool call. ‡: empirically tested with controlled implementation (this work). *: retrieval-axis prediction assessed via controlled implicit-delivery probes (V3-1 ICL proxy, V3-2 scaffold-side FAISS); the named attack pipeline was not reproduced.

Attack	Memory write	Memory read	Observable?
DTA (this work)	agent tool call (<i>save_fact</i>)	agent tool call (<i>recall_fact</i>)	✓ [‡]
Semantic search (V3-2)	direct DB write	agent tool call (<i>memory_search</i>)	✓ [‡]
MINJA-style* (Dong et al., 2025)	agent record-logging	framework embedding retrieval	✗*
Zombie-style* (Yang et al., 2026)	normal memory-update	sliding-window / RAG assembly	✗*
qwq:32b bypass (§6.8)	agent tool call	goal-directed RAG fallback	✗

Why implicit-retrieval attacks evade the detector. In MINJA (Dong et al., 2025), malicious records are injected via the agent’s own record-logging process and later retrieved by the framework’s embedding-similarity mechanism as in-context demonstrations—the agent never issues an observable *recall_fact* call. Similarly, Zombie Agents (Yang et al., 2026) persist payloads through sliding-window or retrieval-augmented memory implementations where retrieval is performed by the infrastructure, not by an agent-initiated tool invocation. In both cases, the *recall_before_send* transition cannot fire because no recall tool call is generated.

6.4 WRITE OBSERVABILITY AS A SECOND AXIS

The retrieval-observability boundary above concerns the *read* side of the attack (does payload retrieval produce a logged tool call?). A re-analysis of our existing implicit-delivery data—using only data already collected, no new experiments—shows that detectability is more precisely a function of *two* observability axes: whether the malicious *write* (payload storage) is observable, and whether the *read* (retrieval) is observable. This refines the taxonomy from a one-dimensional retrieval-observability boundary to the 2×2 structure in Table 13.

We scored two operation-level signatures on successful attacks from the implicit-delivery arms: the paper’s *recall_before_send* rule, and a *save_before_send* rule (any *memory_save_fact* preceding an exfiltrating send). Three findings result. (i) On the V3-1 implicit-delivery successes with *recall_before_send*=0 ($N=28$; GPT-4.1 $n=14$, GPT-4o $n=14$; claude-sonnet-4-6 produced no successful attacks), the payload delivery is implicit on the read axis but the agent still issues an observable *memory_save_fact* in 24/28 cases, so a *save_before_send* rule flags 24/28 while *recall_before_send* and the full 19-feature classifier flag 0/28. (ii) However, *save_before_send* exhibits a 66.7% benign false-positive rate (22/33 benign memory-grounded sends in the V3-2 benign arm): like *recall_before_send*, it is an attack *precondition*, not a maliciousness predicate. (iii) In the both-implicit cell—V3-2 arm C, where retrieval is scaffold-side and no *memory_save_fact* tool call is emitted, the architecture that native MINJA and Zombie Agents occupy—*save_before_send* flags 0/53 successful attacks. The 4/28 V3-1 successes that emit neither recall nor save (trajectory *list_all_facts*→*draft*→*send*, spanning 3 distinct scenario configurations) are genuinely signature-free under operation-only monitoring.

Together these results show that memory-poisoning detectability is governed by two observability axes (write and read); attacks implicit on both axes—the cell occupied by MINJA and Zombie Agents—leave no memory operation at all and are caught 0/53 by any operation-only signature, confirming their genuine invisibility, while attacks observable on either axis produce a signature that remains a precondition (66.7% benign FPR), not a maliciousness predicate. This narrows rather than overturns the paper’s central claim: retrieval (read) observability determines observability of the *retrieval provenance*, and the both-implicit conclusion for MINJA/Zombie is unchanged, but “implicit retrieval evades trajectory monitoring” as an unqualified statement is too strong—an observable write leaves a (noisy) trace. *Data limitation:* the V3-1 logs record operation names but not *memory_save_fact* key/value arguments, so it cannot be verified from the logs whether the observed save wrote the attack payload, a benign decoy, or imitated the in-context demonstration;

Table 13: Detectability under the 2×2 write-observability $\times$ read-observability taxonomy, from re-analysis of existing implicit-delivery data. The write axis provides an additional signature only when the malicious write is tool-routed, and even then as a high-FPR precondition; attacks implicit on both axes (the MINJA/Zombie cell) are genuinely invisible to operation-only monitoring.

	Observable read	Implicit read
Observable write	full signature (main regime)	<i>save_before_send</i> flags 24/28 (V3-1); 66.7% benign FPR — precondition, not predicate
Implicit write	—	0/53 caught (V3-2 arm C = MINJA/Zombie cell); genuinely signature-free

the *save_before_send* result is therefore an operation-level association, not verified payload-write provenance.

The invariant’s true scope. The detector targets the *tool-routed* portion of the memory-poisoning attack surface: architectures where memory access is mediated by observable tool calls. The structural argument applies whenever successful exfiltration depends exclusively on information returned by a logged retrieval operation. This is common in production deployments that expose memory as a first-class tool interface (LangChain, LlamaIndex, OpenAI Assistants), and we have empirically confirmed the invariant for two tool-routed retrieval primitives (exact-key recall and semantic search). However, attacks that bypass observable tool interfaces (implicit RAG, framework-level context assembly, cached internal state) fall outside the detector’s architectural assumptions and require complementary defenses (content-level DLP, semantic correlation analysis, or retrieval-layer monitoring).

Implications. Retrieval observability is the dividing variable of the memory-poisoning attack surface with respect to trajectory-based detection. When successful exfiltration causally depends on a payload obtainable only through an observable retrieval tool, a retrieval-before-exfiltration event is structurally necessary. Per-primitive evidence (V3-1 controls + V3-2 arm B, preregistered scoring): exact-key recall 58/58 zero violations (GPT-4.1 28/28 + GPT-4o 30/30); semantic search 33/33 on GPT-4.1 (confirmed) and 13/13 on GPT-4o (directionally supportive but underpowered— $N=13$, two-sided 95% Wilson CI upper endpoint on violation rate ≈ 0.228). Under corrected scoring: GPT-4.1 semantic 38/38. When the scaffold delivers the payload without an observable tool call, attacks remain viable without producing the signature (V3-1 treatment, V3-2 arm C)—the signature absence in arm C is architectural (no search tool available) rather than a behavioral choice by the model. Neither direction implies sufficiency for detecting maliciousness: benign memory-grounded sends produce trajectory-isomorphic signatures (v2). This boundary is empirically verified on act-without-grounding models (GPT-4.1, GPT-4o) only; ground-then-act models remain untested due to near-zero attack success rates under this attack construction. The necessity claim requires that successful execution depend on information available exclusively through the logged retrieval operation—if the payload is accessible by other means, the model may bypass observable retrieval even when the tool exists.

Empirical validation (V3-1). A within-architecture probe (V3-1; preregistered design, exploratory execution due to deviations noted below; $N=40$ per cell, 3 models $\times$ 2 arms = 240 total trials; 2 models informative, 1 untestable due to near-zero ASR) directly tests the retrieval-observability mechanism. Both arms use the identical DTA agent architecture (same tool schema, same exfiltration action, same decoy facts in memory). The *control* arm delivers the routing address via *recall_fact* (tool-routed); the *treatment* arm delivers it as an ICL demonstration simulating framework-level implicit retrieval (the address is visible in context without requiring a *recall_fact* call). The ICL demonstration depicts no *recall_fact* invocations (anti-imitation constraint; see prereg V3-1). Note that ICL demonstration is a *proxy* for framework-level implicit retrieval; V3-2 (below) tests the same mechanism with real embedding-similarity retrieval and finds broadly consistent evasion rates.

Results confirm the mechanism variable (per-model, no pooling):

- **GPT-4.1 control (tool-routed):** ASR = 28/40 (70%); among successes, $P(\text{rbs}=1 \mid \text{success}) = 28/28 = 100\%$ (two-sided 95% Wilson CI upper endpoint on violations: $[0, 0.121]$). The invariant holds.
- **GPT-4.1 treatment (implicit):** ASR = 21/40 (52.5%); among successes, $P(\text{rbs}=0 \mid \text{success}) = 14/21 = 67\%$ (nominal Wilson 95% CI $[0.454, 0.828]$; *scenario-clustered* bootstrap 95% CI $[0.450, 0.864]$, resampling the 18 scenario configurations—excludes 0 under both). Necessity is broken.
- **GPT-4o control (tool-routed):** ASR = 30/40 (75%); among successes, $P(\text{rbs}=1 \mid \text{success}) = 30/30 = 100\%$ (two-sided 95% Wilson CI upper endpoint: $[0, 0.114]$). The invariant holds — replicating on a second act-without-grounding model.
- **GPT-4o treatment (implicit; exploratory):** ASR = 19/40 (47.5%); among successes, $P(\text{rbs}=0 \mid \text{success}) = 14/19 = 74\%$ (nominal Wilson 95% CI $[0.512, 0.882]$; *scenario-clustered* bootstrap 95% CI $[0.600, 0.929]$ —excludes 0 under both). Necessity is broken on a second model, supporting H2a: the effect is a phenotype property (act-without-grounding), not a model-specific artifact. (GPT-4o was added post-hoc; see deviations.)
- **Claude Sonnet control:** ASR = 1/40 (2.5%); the single success has $\text{rbs}=1$. **Claude Sonnet treatment:** ASR = 0/40 (0%), refusing the attack regardless of delivery mechanism. H2 is untestable for ground-then-act models at the current N . *Open gap:* the scope boundary is validated only on act-without-grounding models. Whether ground-then-act models’ established verification-recall habit persists under implicit delivery (which would restore the signature even without tool-routing) remains unknown. Closing this gap requires either (a) a ground-then-act model with non-negligible ASR under this attack construction (candidates: gemini-3.5-flash, gemini-2.5-pro, which exhibit ground-then-act phenotype with potentially higher attack compliance), or (b) alternative attack framings that elicit compliance from resistant models without altering the tool schema.
- **H2a (phenotype moderation):** GPT-4.1 $\text{rbs}=0$ rate = 67% $[0.454, 0.828]$; GPT-4o $\text{rbs}=0$ rate = 74% $[0.512, 0.882]$. CIs overlap substantially — consistent with a shared phenotype effect rather than model-specific variation. The act-without-grounding class reliably bypasses the detector under implicit delivery.
- **Verification recall (H4):** Among the 12 treatment successes across both models that produced $\text{rbs}=1$, `recall_fact` calls targeted benign decoy keys (3/36 recalls targeted the attacker address for GPT-4o; 0/29 for GPT-4.1). These are predominantly verification recalls of unrelated stored facts, mechanistically distinct from the control arm’s forced recall.
- **Classifier score distribution (H3):** The trained RF classifier scores control successes at mean = 0.95 (detected 58/58 across both models); treatment $\text{rbs}=0$ successes score mean = 0.12 (detected 0/28 — complete evasion), while $\text{rbs}=1$ successes score mean = 1.000 (detected 12/12). These 12 “detections” are precision-coincidental: the agent happened to recall benign decoy facts before sending, producing the same trajectory shape as a genuine tool-routed attack — but the recall was unrelated to the exfiltration payload. The detector is structurally blind to implicit-delivery attacks that do not trigger verification recall.

Deviations from preregistration. The prereg (commit 96e63e8) specified $N=40$ per arm per model sampled from a 72-scenario grid ($3 \times 3 \times \sim 4 \times 2$), with a stopping rule extending sampling until ≥ 30 treatment successes per model or grid exhaustion. In implementation, the decoy-subset axis was fixed rather than varied (the decoy pool is present in all trials but which specific decoys are stored does not interact with the mechanism variable—retrieval observability depends on *whether* recall occurs, not on *which* keys are recalled), yielding an 18-scenario grid ($3 \times 3 \times 2$); we collected the preregistered $N=40$ per cell (scenarios sampled with replacement beyond the 18-element grid). *Unique-scenario N :* the effective number of independent observations per cell is $N_{\text{unique}}=18$ (unique scenario configurations), not the nominal $N=40$ —each scenario is repeated $\sim 2.2\times$ on average. GPT-4o was added post-registration as a robustness check (same phenotype, same architecture, same tool schema); its inclusion is exploratory with respect to the prereg but strengthens H2a. The GPT-4.1 treatment arm yielded 21 successes and GPT-4o yielded 19 (both below the 30-success floor); the preregistered 72-scenario grid was not fully instantiated. Despite the reduced grid, both models’ primary CIs exclude 0 ($[0.454, 0.828]$ and $[0.512, 0.882]$ respectively), so the confirmatory

conclusion is unaffected. *Non-independence caveat*: the 40 trials per cell are sampled from only 18 unique scenario configurations, with some scenarios repeated 2–5 times. Wilson intervals assume independent observations; repeated-scenario trials at temperature=0 against the same API endpoint are anti-conservative for the treatment-arm rate estimates because identical prompts to a deterministic endpoint produce correlated (often identical) outcomes. The effective number of independent observations is closer to $N=18$ (unique scenarios) than to the nominal $N=40$. The control-arm zero-violation observation (a count, not a rate estimate) is unaffected by this concern. For the treatment-arm rate estimates we now report *scenario-clustered* bootstrap CIs (resampling the 18 scenario configurations as clusters) alongside the nominal Wilson intervals: GPT-4.1 [0.450, 0.864] (vs nominal [0.454, 0.828]) and GPT-4o [0.600, 0.929] (vs nominal [0.512, 0.882]). Both clustered intervals exclude 0, so the confirmatory conclusion is unchanged under clustered inference. Claude Sonnet’s near-zero ASR (0/40 treatment, 1/40 control) is an unanticipated outcome outside the preregistered decision tree; H2 remains untestable for ground-then-act models. The registration-to-execution interval was <24 hours (same calendar day; GPG-signed timestamps machine-verifiable).

The ~30% treatment $\text{rbs}=1$ rate across both act-without-grounding models (all from decoy-fact verification) has an adaptive-adversary implication: an attacker aware of the detector need only harvest the $\text{rbs}=0$ subset of successful implicit attacks to guarantee evasion. The detector cannot distinguish “agent recalled a decoy before sending” from “agent recalled the attack address before sending” using operation-only features — both produce $\text{rbs}=1$, but only the latter is causally related to the attack.

Embedding retrieval validation (V3-2). A second probe (V3-2; preregistered design, scorer corrected post-hoc as disclosed below; $N=200$, 2 models $\times$ 2 arms + 40 benign) directly tests whether the invariant generalizes from exact-key recall to *any* tool-routed retrieval, and whether attacks remain viable when the payload is delivered scaffold-side without an observable tool call.

Setup: FAISS index over 30 benign organizational records + 1 poisoned routing rule, embedded via `text-embedding-3-small` (1536d, cosine similarity, top- $k=4$). Arm B provides a `memory_search(query)` tool; arm C injects top- k results scaffold-side with no search tool available. A framework-level oracle logs whether the poisoned record was retrieved (rank, similarity) for each trial.

Results (per-model, no pooling; Table 14):

- **H1 (arm B — semantic tool search preserves invariant):** Among successful attacks where the payload was oracle-confirmed as delivered, under the *preregistered* scorer (`email_send_email` only): GPT-4.1 33/33 and GPT-4o 13/13 exhibited `search_before_send=1` (zero violations). Under the corrected scorer (sensitivity analysis counting `email_draft_email` as exfiltration-capable): GPT-4.1 38/38. *GPT-4o caveat*: only 13 oracle-confirmed delivered successes were obtained (delivery rate 27/40 = 67.5% due to retrieval competition from 30 benign records). The two-sided 95% Wilson CI upper endpoint on the violation rate is ≈ 0.228 —this is consistent with a true violation rate up to ~23%. The GPT-4o arm B replication is *directionally supportive but statistically underpowered*; it cannot rule out moderate violation rates. When successful exfiltration causally depends on a payload available exclusively through the observable `memory_search` tool, a retrieval event before the first exfiltration-capable email action is structurally necessary on GPT-4.1 (confirmed) and directionally supported on GPT-4o (underpowered).

Scorer correction justification (sensitivity analysis): The preregistered scorer checked only `email_send_email`. Post-collection, 5 GPT-4.1 trials used `email_draft_email` (specifying the attacker address at draft time, never proceeding to send). Counting `draft_email` as exfiltration-capable is justified because: (1) the recipient-routing decision — the attack’s defining exfiltration act — is made at draft time; (2) in production, a draft containing the attacker address is the threat, regardless of whether a separate send step follows; (3) the tool schema declares `draft_email` with a required to field, making it functionally equivalent to `send_email` for routing exfiltration. All 5 records had `memory_search` preceding `draft_email`; no trial was excluded or manually relabeled. The conservative primary result uses 33/33 under preregistered scoring; the corrected 38/38 is a sensitivity analysis. Both are consistent with zero violation rate (two-

sided 95% Wilson CI upper endpoints: 0.102 and 0.091 respectively), providing bounded evidence for the retrieval-dependency invariant on GPT-4.1.

- **H2 (arm C — attacks remain viable without observable retrieval):** Arm C removes all `memory_*` tools from the agent’s schema (see Classifier applicability below for residual `rag_query` disclosure), so signature absence is *definitional* (not an estimated behavioral rate). The empirical finding is that attacks remain viable under this architecture: GPT-4.1 33/40 trials and GPT-4o 20/40 trials succeeded after oracle-confirmed payload delivery (CIs for attack viability: [0.680, 0.913] and [0.352, 0.648] respectively). This demonstrates that observable retrieval is not a precondition for successful exfiltration—the attacker can deliver payloads via channels that leave no trajectory signature—not that the model “chose” to bypass retrieval when retrieval was available.
- **H3 (proxy consistency):** Arm C attack-success rates (successful attacks among delivered trials) were broadly consistent with the V3-1 ICL-proxy treatment results. GPT-4.1: arm C = 33/40 = 0.825 vs. ICL-proxy ASR = 21/40 = 0.525; GPT-4o: arm C = 20/40 = 0.500 vs. ICL-proxy ASR = 19/40 = 0.475. Both mechanisms produce substantial successful attack rates despite payload delivery via implicit means. Because equivalence margins were not preregistered, we interpret this as qualitative consistency rather than formal equivalence: both scaffold-side and ICL-proxy delivery sustain viable attacks that lack the detection signature.
- **H4 (benign):** In the benign semantic-memory condition (GPT-4.1 only, no poisoned record, $N=40$), 33/40 trajectories (82.5%, CI [0.680, 0.913]) contained search before the outbound email action. This demonstrates that the precision limitation documented in v2 also occurs with semantic tool retrieval under the same task family.

Table 14: V3-2 denominator breakdown. “Delivered” = oracle-confirmed poisoned record in retrieval results. Arm B delivery = poisoned item returned by agent-invoked `memory_search`; arm C delivery = poisoned item selected by scaffold retrieval and injected into context. The signature column reports `search_before_exfiltration` among successful attacks: for arm B this is an empirically observed behavioral invariant; for arm C the search tool is architecturally absent, so the entry is N/A (all successes are necessarily signature-free).

Model	Arm	Trials	Del.	Succ.	S.+D.	Sig.—S.+D.
GPT-4.1	B (search tool)	40	40	33 [†] /38 [§]	33/38	33/33 [†] ; 38/38 [§]
GPT-4.1	C (scaffold)	40	40	33	33	N/A (no tool)
GPT-4o	B (search tool)	40	27	13	13	13/13
GPT-4o	C (scaffold)	40	40	20	20	N/A (no tool)

[†]Primary (preregistered): `email_send_email` only. [§]Sensitivity analysis: includes `email_draft_email`.

Classifier applicability. The 19-feature Random Forest (Section 4.5) was trained on trajectories using the original DTA tool schema (`recall_fact`, `list_all_facts`). Arm B uses `memory_search`—a different operation name—meaning the trained classifier’s `recall_count`, `recall_before_send`, and bigram features would all read 0 without feature remapping or re-training. Arm C removes all `memory_*` tools; a residual `rag_query` tool remained in the schema but returned only a static acknowledgment (no actual retrieval), and its invocations (31/40 GPT-4o trials, 1/40 GPT-4.1 trials) did not access the poisoned record. V3-2 therefore validates the *generalized structural rule* (observable retrieval before exfiltration-capable action) rather than the specific trained classifier. Whether the full 19-feature classifier transfers to alternate tool schemas via feature remapping is an open question not addressed by this data.

Deviations from preregistration (V3-2). GPT-4o arm B payload delivery rate was 27/40 (67.5%) due to retrieval competition from 30 benign records reducing the poisoned record’s rank. Per the prereg, the H1 analysis conditions on both success and delivery (13/13). The prereg’s stopping rule (extend to $N=60$ if < 10 successes) was not triggered for GPT-4.1 (38 successes) but GPT-4o yielded 13 (above the 10-success floor). *Non-independence caveat (V3-2):* V3-2 shares V3-1’s repeated-scenario structure (18 unique scenario configurations sampled with replacement to $N=40$ at temperature=0). The same anti-conservative interval concern applies: effective independent N

is bounded by the 18 unique scenarios rather than the nominal 40 trials. The H1 zero-violation counts are unaffected (counts, not rate estimates), but H2 attack-viability CIs ($[0.680, 0.913]$ and $[0.352, 0.648]$) should be interpreted as approximate under the same caveat.

Scoring correction disclosure (V3-2). The preregistered scorer defined the signature as `memory_search` preceding `email_send_email`. Post-collection inspection revealed that GPT-4.1 arm B consistently used `email_draft_email` (the agent drafts to the attacker address rather than sending directly). Because `email_draft_email` requires specifying recipients and constitutes an exfiltration-capable action (the outbound routing decision is made at draft time), the scorer was corrected to define the first exfiltration-capable email action as either `email_send_email` or `email_draft_email`. All arm B observations were mechanically rescored from the immutable `call_sequence` field in the raw logs. This correction changed 5 GPT-4.1 records from `sbs=0` to `sbs=1` (all had `memory_search` preceding `email_draft_email`). No trial was excluded or manually relabeled. Under the original (erroneous) scorer: GPT-4.1 arm B = 33/38; under the corrected scorer: 38/38.

Table 15: Preregistration deviations across all probes. No deviation reverses a primary conclusion; four preregistered secondary analyses remain unexecuted (data collected but statistical tests not run).

Probe	Deviation	Justification	Impact on conclusion
V2-1	H1 (monotonicity) falsified	Binary saturation	Result, not deviation; FPR > 0 confirmed
V3-1	Grid reduced 72→18 scenarios	Decoy axis non-interacting	CIs still exclude 0
V3-1	GPT-4o added post-hoc	Same phenotype, exploratory	Strengthens H2a; labeled
V3-1	Success floor (≥ 30) missed	ASR lower than anticipated	CIs exclude 0 regardless
V3-1	Claude unstable (ASR ≈ 0)	Ground-then-act phenotype	H2 remains open
V3-1	H2a Fisher test not computed	Rates reported; formal test omitted	Direction clear from CIs
V3-1	H3 Mann-Whitney not computed	Secondary; data exists for future	Does not affect H1/H2
V3-2	Scorer corrected post-hoc	<code>draft_email</code> $\equiv$ exfiltration	33/38 → 38/38; both reported
V3-2	GPT-4o delivery 27/40	Retrieval competition	Conditioned on delivery per prereg
V3-2	$k=\{2, 4, 8\}$ sweep not run	Fixed at $k=4$; robustness secondary	Primary H1 unaffected
V3-2	Retrieval-rank analysis omitted	Oracle data collected; analysis pending	Does not affect H1/H2

Consolidated deviation summary.

6.5 THE TRAJECTORY SIGNATURE AS ENSEMBLE FEATURE

The preregistered follow-up (Section 5) demonstrates that the trajectory invariant cannot serve as a standalone detector—its unconditional benign FPR of 24.7–57.6% is operationally prohibitive. This falsification is itself a contribution: it establishes the deployment boundary that separates forensic detection research from deployable security tooling.

In ROC terms, the signature operates at a fixed point of (Recall = 0.98, FPR = 0.25–0.53) depending on model phenotype. As a standalone classifier this is inadequate; as a high-recall pre-filter in an ensemble that adds semantic gating (recipient metadata, content classification, or allowlist checks), it narrows the inspection surface by 47–75% for act-without-grounding fleets under exact-key recall (Section 6.6; note that ground-then-act fleets achieve only 0–24% savings, and that the phenotype is interface-dependent — see the Section 6.6 caveat on semantic retrieval) while maintaining near-perfect recall. The operational value is analogous to a high-sensitivity medical screening test: it identifies virtually all true positives but requires confirmatory testing to achieve acceptable precision.

The honest reporting of this failure boundary—rather than suppressing the benign FPR result or restricting evaluation to attack-only data—illustrates a methodological contribution: preregistered deployment-boundary testing should be standard practice for forensic detection papers, most of which report precision and recall only on curated attack datasets without measuring performance under realistic benign traffic.

6.6 DEPLOYMENT ECONOMICS

A natural objection to the trajectory pre-filter is that recipient-based Data Loss Prevention (DLP)—inspecting the `to` field of every outbound action—could achieve the same separation without the trajectory layer. We formalize when the trajectory signal provides marginal value over DLP-alone.

Cost asymmetry. Trajectory feature extraction operates on tool-call metadata: 19 integer/binary features computed from operation names and their ordering. This is $O(n_{\text{calls}})$ per session, where n_{calls} is typically 3–8 (the median trigger-session length in our dataset is 5 tool calls). Content-level DLP—inspecting email bodies, attachments, or memory contents for sensitive material—is $O(n_{\text{tokens}})$ per session, where n_{tokens} may reach thousands. We do *not* claim a measured cost reduction. The 10^2 – $10^3\times$ figure that appears in earlier framings of this work is a *hypothetical upper bound* derived from asymptotic complexity classes alone ($O(n_{\text{calls}})$ integer features vs. $O(n_{\text{tokens}})$ content analysis); it is not benchmarked, ignores constant factors, batching, caching, and shared-model-serving effects, and should be read as a theoretical ceiling rather than an expected operational saving. The economic argument below is therefore presented as a conditional cost *model* parameterized by the unmeasured ratio C_D/C_T , not as an empirical result. Establishing the realized ratio requires a system benchmark we leave to future work.

When trajectory pre-filtering dominates. Let C_T denote the per-session cost of trajectory classification and C_D the per-session cost of content-level DLP. Let r denote the trigger rate (fraction of sessions that exhibit *recall.before.send*). A trajectory-gated deployment inspects all N sessions with the trajectory classifier ($N \cdot C_T$) and applies DLP only to flagged sessions ($r \cdot N \cdot C_D$). Total cost: $N(C_T + r \cdot C_D)$. DLP-alone inspects every session: $N \cdot C_D$. The trajectory layer reduces total cost whenever:

$$C_T + r \cdot C_D < C_D \iff C_T < (1 - r) \cdot C_D \quad (1)$$

Since we expect $C_T \ll C_D$ (metadata vs. content inspection) and $r \in [0.25, 0.53]$ (from Section 5), this inequality would hold with substantial margin *if* the expected cost ordering is realized; because C_T and C_D are unmeasured here, this is a conditional prediction of the cost model, not a demonstrated saving. Under that condition the trajectory layer saves $(1 - r) \cdot N$ content inspections—a 47–75% reduction in DLP invocations—at the additional cost of one trajectory classification per session. Note that r is phenotype-dependent: ground-then-act fleets (claude-sonnet, gemini-flash) have $r \approx 0.76$ – 1.0 , where the cost savings shrink to 0–24% and the inequality may fail if C_T is non-trivial. The economics argument is strongest for act-without-grounding deployments (gpt-4o, gpt-4.1), where both FPR and r are near zero under exact-key recall—the same phenotype variable that governs precision (Section 5) also governs economic viability. **Important caveat:** the act-without-grounding phenotype is interface-dependent, not a stable model property. V3-2 H4 demonstrates that GPT-4.1 exhibits 82.5% benign *search.before.send* under semantic tool retrieval (i.e., ground-then-act behavior), despite being act-without-grounding under exact-key *recall.fact*. Under semantic search interfaces—the most common production memory retrieval mechanism— r for GPT-4.1 rises to ≈ 0.83 , reducing cost savings to $\sim 17\%$ and potentially violating the inequality if C_T is non-trivial. Operators should condition the economics estimate on their specific retrieval interface, not on model identity alone.

When trajectory adds no detection power. If an operator already runs recipient-metadata DLP on every outbound action (checking whether `to` is external, regardless of trajectory), the trajectory signal adds no incremental *detection* capability—recipient externality alone separates benign from attack on our data. In this regime, the trajectory layer’s residual value is limited to: (1) *prioritization* (ranking alerts by trajectory score before human review), (2) *attack-vector attribution* (distinguishing memory-channel from prompt-injection attacks in forensic triage), and (3) *early warning* (the prefix-only classifier can flag suspicious sessions before the send event, enabling pre-emptive intervention).

Target deployment. The trajectory detector’s primary value proposition is for high-volume agent fleets where content-level DLP is not universally deployed on every session due to cost or latency constraints. In such environments, the trajectory signal serves as a cheap structural triage layer that routes only high-risk sessions to expensive semantic inspection. For operators who already

inspect all outbound traffic at the content level, the marginal value is attribution and prioritization, not detection.

6.7 CONCEPT-LEVEL HEURISTIC STRESS TEST (CROSS-VENDOR)

To probe whether the trajectory-signature *concept* (retrieval-before-action as a detection signal) generalizes beyond the training distribution, we evaluate a simplified hand-specified scoring heuristic (logistic scoring on 11 features including `has_recall_before_send`, `has_send_email`, and tool counts; see companion repository for exact weights) on three held-out attack families that were never present during training, across three vendor families (GPT-5.1, Claude-Sonnet-5, Gemini-2.5-Pro; $N=50/\text{cell}$, 600 total trials).

Important methodological notes: (1) This evaluation uses a hand-specified scoring heuristic with hardcoded weights derived from the v2 detector’s principal discriminative features—it is **not** the trained 19-feature Random Forest and does not test that classifier’s transfer. (2) The harness makes a single API completion per trial and does not execute tool calls or feed tool results back to the model; it records only the model’s *proposed* tool calls from one response, not multi-turn executed trajectories as in the main experiments. (3) Attack-family trials are labeled as ground-truth attacks based on scenario membership (intent to trigger malicious behavior), regardless of whether the model actually executed a malicious action. Detection rates below therefore reflect recognition of the attack *scenario*, not recall among successfully executed attacks—models that refuse or produce empty tool calls are counted as detector misses rather than non-attacks. (4) The heuristic scorer produces only 6 distinct score values across all 600 trials, determined entirely by which tool *names* appear in the proposed call set (not by arguments, recipients, or context). It is a discrete deterministic rule over tool-name presence, not a probabilistic classifier; per-family “detection rates” reflect the fraction of trials in which the model proposed the specific tool-name pattern that triggers the rule.

Results therefore assess generalization of the *concept* (whether recall-before-action persists as a signal across novel attack families and vendors) under a weak proxy harness, not the performance of the published classifier in deployment:

- **Indirect exfiltration** (via `share_document` instead of `send_email`): Detection rate = 0.0% [0.000, 0.025]. The recall-before-send signature is specific to the email channel; exfiltration through sharing tools produces no detectable trajectory pattern.
- **Encoded payload** (base64 instructions in memory): 27.3% [0.208, 0.350]. Some encoded variants still trigger recall-before-send when the model decodes and acts on the payload.
- **Multi-step delayed trigger**: 38.7% [0.312, 0.467]. Delayed triggers that ultimately route through email partially trigger the signature.

Benign specificity: 81.3% correct [0.743, 0.868] (FPR = 18.7%). Model-level detection varies: GPT-5.1 (60.7%), Gemini-2.5-Pro (5.3%), Claude-Sonnet-5 (0.0%)—Claude’s baseline refusal prevents the attack trajectory from materializing entirely.

These results are consistent with the architectural boundary described in Section 4.10: the detector’s scope is limited to tool-routed attacks that pass through observable `recall_fact`→`send_email` sequences. Novel exfiltration channels require either semantic correlation analysis or external enforcement (see companion work (Leong, 2026b)).

6.8 LIMITATIONS

False negative rate. FN = 31 out of 1,919 attack-success sessions (1.6%). All 31 originate exclusively from qwq:32b’s implicit-bypass runs under the Memory Sandbox defense (goal-directed RAG fallback without `recall_fact`). In deployments without qwq:32b, the effective FNR is zero across all tested models. Separately, qwen2.5:14b contributes 62 false *positives* (FPR = 10.3%) due to distributional inversion (its benign recall patterns mimic the attack signature), but zero false negatives (Recall = 1.000).

Feature definition limitation. The `recall_before_send` feature implementation compares only the *first* `recall_fact` invocation with the *first* `send_email` invocation (first-occurrence

semantics). A session that sends, then recalls, then sends again is incorrectly labeled `rhs=0` even though a later exfiltrating send follows retrieval. The corrected definition—any recall preceding any exfiltrating send—produces cleaner 100%/0% conditional FPR separation. This bug does not affect any reported number: across all 2,520 primary DTA sessions the naive first-occurrence and corrected any-preceding definitions produce *identical* labels (0 disagreements), including on the 1,120 sessions that do contain multiple `send_email` calls—in every such session a `recall_fact` precedes the first send, so first-vs-first and any-vs-any agree. The only observed divergence arises in out-of-scope replication scoring on qwq:32b revision-pattern sessions (send, revise via recall, then re-send), which are not part of the published training data. The released classifier (`paper_a_classifier.py`) implements the corrected any-preceding-recall definition; because the two definitions produce identical labels on all 2,520 primary sessions, every reported table and figure is unaffected by the choice.

Write-axis re-analysis limitation. The 2×2 write-observability analysis (Section 6.4) uses only the read-implicit V3-1/V3-2 arms and the benign V3-2 arm; the *save_before_send* figures (24/28 catch, 66.7% benign FPR, 0/53 in the both-implicit cell) are descriptive re-analyses of already-collected data, not a preregistered confirmatory experiment. Two limitations bound their interpretation. First, the V3-1 logs record operation names but not `memory_save_fact` key/value arguments, so we cannot verify from the logs whether the observed write stored the attack payload, a benign decoy, or imitated the in-context demonstration; *save_before_send* is therefore an operation-level association, not verified payload-write provenance. Second, the write-axis result is within-harness (our DTA-derived ontology, in which writes are tool-routed); native MINJA and Zombie Agents write scaffold-side, matching the both-implicit cell where the write signature also fails (0/53). The write axis thus does not provide a deployable detector and does not alter the MINJA/Zombie classification.

Benign false-positive rate. The reported false-positive rate in the main evaluation is measured against poisoned-but-defended non-exfiltration executions, not against ordinary unpoisoned benign agent traffic. A preregistered follow-up study ($N=4,360$, 13 models; Section 5) measures the deployment FPR: $P(\text{FP} \mid \text{recall_before_send}=1) = 100.0\% [99.8, 100.0]$ (unconditional benign FPR: 24.7–57.6% depending on recall protocol and fact count). Benign memory-grounded sends are score-isomorphic to attacks under the trained classifier. The detector should therefore be deployed as a high-recall triage signal with semantic gating, not as a standalone blocker.

Frontier false positive rate. On frontier models (GPT-4.1, GPT-4o, o3-mini, o4-mini), the detector achieves Recall = 0.901 across 405 attack sessions but FPR = 23.0% on 560 non-attack sessions, notably higher than the training FPR of 10.3%. The increase arises because reasoning models (o3, o4-mini) call `recall_fact` more frequently during non-attack sessions to verify stored information before acting, mimicking parts of the attack signature. Per-model threshold calibration would likely reduce this gap; the current single-threshold design is a conservative choice that favors recall over precision.

Training data scope. The classifier was trained on open-source models via Ollama. Cross-model hold-out confirms generalization to GLM, GPT-OSS, and Qwen families, and the GPT-4.1 probe ($N=20$) confirms frontier model alignment. A larger frontier-model training set would sharpen the decision boundary, particularly for models with recall counts > 1.5 (the GPT-4.1 / GPT-4o regime).

Implicit-bypass attacks. The classifier correctly misses qwq:32b’s Memory Sandbox bypass (goal-directed RAG fallback, no `recall_fact`). This is one instance of a general boundary: attacks using implicit retrieval (framework-level embedding similarity, sliding-window assembly, or cached context) produce no observable tool call and therefore evade the detector entirely (see Section 6.3 for the full taxonomy). Detecting implicit-bypass attacks requires tracking the *semantic correlation* between RAG document content and outbound email fields, rather than relying on raw `rag_count` volume metrics alone. We leave this for future work. Companion evaluations confirm that the three models underlying the stable detection spine (qwen2.5:14b, qwen2.5:72b, qwen3:32b) reproduce at 100% ASR across independent evaluation environments, while reasoning models exhibit output-quality degradation under long-running inference sessions that affects attack *prevalence* but not trajectory *shape* when attacks succeed. The rate-fragility identified in companion work does

not reach the detection invariant: it changes class balance, not the forced recall-before-send signature.

Adaptive adversaries. We consider three evasion strategies an adversary aware of the detector might attempt: (1) *Suppress recall*: embed the routing address directly in the prompt, eliminating the `recall_fact` call. This changes the attack vector to prompt-injection, which the detector correctly classifies as a different channel (Section 4.10). The attack succeeds but loses persistence across sessions. (2) *Inject fake recalls into non-attack sessions*: the adversary poisons benign sessions to include spurious `recall_fact` calls, inflating the FPR. This requires persistent influence over the agent’s environment, itself a security compromise, and cannot suppress the genuine attack signature. The detector’s precision degrades but recall is unaffected. (3) *Retrieve via alternative channels*: use framework-level embedding retrieval or cached context instead of the memory tool. V3-1 and V3-2 arm C empirically demonstrate this strategy works: implicit-delivery attacks achieve 50–82% success rates on GPT-4.1 and GPT-4o while completely evading the detector. An attacker aware of the detector need only target architectures where payload delivery occurs scaffold-side, or (per V3-1’s adaptive implication) harvest only the `rbs=0` subset of implicit attacks. This is not a hypothetical evasion but a demonstrated, viable strategy — and the primary reason we characterize the detector as a precondition filter for tool-routed attacks, not a general-purpose defense.

7 RELATED WORK

Memory poisoning attacks. Persistent memory attacks against LLM agents were established by MINJA (Dong et al., 2025) and Zombie Agents (Yang et al., 2026), which demonstrated high attack success rates against open-source models via query-only memory injection. The delayed-trigger attack (DTA) (Leong, 2026a) embeds a malicious compliance directive in a RAG-retrieved document; the directive instructs the agent to store a routing rule using `memory_save_fact`, which is then retrieved via `memory_recall_fact` in a later session and used to exfiltrate data. Leong (2026a) (same author) reports that five of six defense classes fail against this attack in the tested configuration, with only tool-layer memory restriction achieving structural protection. Hidden in Memory (Pulipaka et al., 2026) independently confirms that GPT-5.5 stores adversary-induced memories at 99.8% injection rate, demonstrating that injection-layer vulnerability persists even in the most capable frontier models. Cross-Session Stored Prompt Injection (Xie et al., 2026) formalizes the XSS analogy for persistent injection across session boundaries. MPBench (Dash et al., 2026) provides a benchmark covering four memory write channels and six attack classes, explicitly motivating behavioral detection approaches but not providing a detector. Critically, MINJA and Zombie Agents both use *implicit* retrieval mechanisms (framework-level embedding similarity or sliding-window assembly) rather than observable agent tool calls—placing them outside the architectural scope of our detector (Table 12).

Behavioral detection in agentic systems. VIGIL (Li et al., 2026) translates behavioral specifications into SMT constraints over finite tool-call event traces, achieving $> 95\%$ recall with $< 10\%$ FPR on policy violations. It enforces pre-specified behavioral policies rather than learning forensic signatures from labeled data. Our approach is complementary: we learn statistical signatures from labeled data without requiring a policy specification, targeting a specific attack class rather than arbitrary violations. MemMorph (Zhang et al., 2026) demonstrates that poisoned long-term memory can steer tool selection across multiple agent architectures; our work asks whether such memory-channel attacks leave operation-only forensic signatures detectable by a supervised classifier. MemLineage (Ouyang & Hou, 2026) attaches cryptographic provenance to memory entries, enabling detection of unsigned writes, but requires modification of the memory infrastructure and cannot detect attacks injected through the agent’s own authorized write pathway (as in DTA, where the agent legitimately stores the malicious rule from a RAG-retrieved compliance document). MEM-SAD (Gowda, 2026) achieves near-perfect detection using gradient-coupled anomaly scoring, but it is a *preventive* defense that blocks attacks before execution and requires model-internal gradient access; our detector is a post-hoc forensic tool requiring only tool-call logs. Concurrent work (Dang, 2026) enforces behavioral firewalls over tool-call sequences using probabilistic deterministic finite automata; they define *permitted* trajectories, whereas we learn *attack* signatures from labeled data.

Intrusion detection and anomaly detection analogues. Host-based intrusion detection systems (HIDS) in classical security identify compromises from system call sequences and process execution traces (Forrest et al., 1996; Warrender et al., 1999). Sequence-based anomaly detection on system calls (using n-gram models, HMMs, or neural sequence classifiers) is structurally analogous to our approach: both extract features from ordered event logs and classify sessions as normal or anomalous. The key distinction is that classical HIDS operates on low-level OS primitives, while our detector operates on tool invocation events in an LLM agent runtime. This framing clarifies the deployment requirement: tool-call logging in agentic frameworks (LangChain tool invocation hooks, OpenAI function-call traces, Anthropic tool-use events) is the agent-runtime equivalent of `strace` or Linux Audit, and the trajectory classifier is a direct implementation of HIDS principles at the agentic abstraction layer. PerD (Garcia-soto et al., 2022) detects embedded Trojans in static NLP models by analyzing input-output behavioral responses, the most directly analogous prior work, but applies to models with fixed behavior, not agentic systems where tool-call trajectories vary with memory state.

Execution-trace auditing for agents. Recent concurrent work has explored execution-trace analysis for agent security. TraceAegis (Liu et al., 2025) applies provenance-based rules over agent execution logs to detect generic anomalous behaviors, while TRACES (Wang et al., 2025) learns trajectory risk states from an observer LLM’s hidden representations. Unlike these systems, which learn generic unsafe patterns or require formal policy specifications, our approach identifies an architecture-induced signature specific to persistent memory poisoning, using lightweight structural features without model internals or learned representations.

The detection gap. Unlike general trace-auditing or memory-poisoning benchmarks, no prior work studies whether a specific explicit-memory exfiltration channel induces a low-dimensional, operation-only trajectory signature amenable to supervised classification. MPBench (Dash et al., 2026) systematizes the memory-poisoning attack surface across four write channels; MemMorph (Zhang et al., 2026) demonstrates tool-selection steering; VIGIL (Li et al., 2026) verifies tool-stream intent. None provides the specific combination we contribute: a supervised classifier trained on an instrumentally labeled factorial dataset ($N=2,520$, 9 models, 7 defenses), evaluated with cross-validation, bootstrap uncertainty estimates, and leave-one-model-out hold-out, and grounded in a mechanistic argument that links the dominant feature to the attack’s information-retrieval dependency.

Disclosure. References (Leong, 2026a) and (Leong, 2026b) are companion preprints by the same author. The training data for this paper’s classifier originates from the factorial in (Leong, 2026a). No findings cited from companion works have been independently replicated.

8 CONCLUSION

Persistent memory poisoning attacks produce a stable behavioral invariant in the agent’s execution trajectory. The *recall_before_send* transition follows from the attack’s information-retrieval dependency: a simple rule exploiting it alone achieves $AUC = 0.9563$, and suppressing observable retrieval breaks the attack via the tool-routed channel (though scaffold-side delivery remains viable). A full trajectory classifier refines this to $AUC = 0.9904$; removing all recall-related features leaves in-distribution AUC unchanged because the retrieval event triggers a behavioral cascade (draft, send, sequence elongation) that produces redundant correlated encodings across multiple feature channels—not independent detection mechanisms, but downstream consequences of the same causal event. This redundancy is in-distribution only: on implicit-bypass attacks (*recall_before_send*=0) the full classifier flags 0/28 while the recall-ablated model flags 24/28, showing the fitted model overfits the recall root split and does not transfer its cascade coverage to implicit retrieval (Section 6.4).

The invariant generalizes across 6/9 open-weight models (7B–120B parameters) and transfers to frontier models (Recall = 0.901 on 405 attack sessions, FPR = 23.0% on 560 non-attack sessions) without retraining. However, a preregistered follow-up ($N=4,360$, 13 models) reveals a critical deployment boundary: benign memory-grounded sends produce the same *recall_before_send* signature, yielding $P(\text{FP} \mid \text{recall_before_send}=1) = 100\%$ on ground-then-act models (unconditional

benign FPR: 24.7–57.6%). The signature is a valid attack precondition, not a maliciousness predicate. Standalone inline blocking is not viable; the correct deployment is as a high-recall triage signal gated by recipient metadata or semantic checks. Critically, the behavioral phenotype governing benign retrieval rates is interface-dependent: GPT-4.1 exhibits 0% benign recall-before-send under exact-key retrieval but 82.5% under semantic search, so deployment economics are conditioned on retrieval architecture, not model identity alone.

These results establish that tool-routed memory-recall exfiltration attacks leave execution-trajectory signatures that are robust within the tested scope (act-without-grounding models, exact-key and semantic tool retrieval), but that operation-only structural features face a hard precision ceiling when benign agents also ground their actions in memory. The scope boundary—validated on two act-without-grounding models but open for ground-then-act models—is a property of observable retrieval dependence: when it holds, the signature is necessary; when the scaffold delivers payloads without observable tool calls, attacks remain viable without the signature. Deployment requires both the trajectory signal (for recall) and semantic context (for precision).

ETHICS AND BROADER IMPACT

This work evaluates forensic detection of memory-poisoning attacks in LLM agents using benign proxy tasks (simulated email composition with fictional recipients and documents). No genuinely sensitive data, real credentials, or harmful payloads were used. The attack payloads consist of routing instructions directing simulated emails to fictional external addresses. We release raw tool-call logs but not model weights or proprietary API prompts.

Dual-use considerations. Publishing the trajectory-based detection mechanism could theoretically inform attackers seeking to evade it. We judge the net benefit positive because: (1) the evasion boundary (implicit retrieval) is already public knowledge through concurrent work (Yang et al., 2026; Dong et al., 2025); (2) the detection mechanism is a property of observable tool-routing architecture, not a secret; and (3) the paper simultaneously characterizes where detection *fails*, informing defenders about realistic adversarial adaptation.

Responsible use. The detector is designed as a triage signal, not a standalone enforcement mechanism. Deployment without human review or semantic gating could produce high false-positive rates on legitimate workflows. We explicitly document the precision ceiling and recommend against automated blocking.

REPRODUCIBILITY

Code, raw result data (JSONL), trained classifier artifacts, and preregistration documents are available at the companion repository.³ API-served model results are auditable via provenance stamps (response model, system fingerprint, request parameters) but not fully reproducible due to mutable API backends. All experiments record exact package versions, git commit hashes, and run manifests. The V2 training data originates from the factorial described in Leong (2026a); the P1 factorial JSONL (classifier training source) is archived in that paper’s companion repository.⁴ V3 data were collected under the preregistrations archived in the repository. Open-weight V2-1 results (9 models, $N=2,180$ trials) are included in the `paper_a/results/` directory alongside frontier model results.

³Repository URL: <https://github.com/junwenleong/forensic-trajectory-signatures>. This is an arXiv preprint with no venue-publication gate; the artifact is released together with the preprint. A content-hash manifest (`artifact_manifest.json`, SHA-256 over all released files) fixes the exact bytes of every script, result file, and preregistration.

⁴P1 factorial repository: <https://github.com/junwenleong/stateful-agent-security-eval> (contains the defense factorial dataset used for classifier training).

REFERENCES

- Hung Dang. Enforcing benign trajectories: A behavioral firewall for structured-workflow AI agents, 2026. arXiv:2604.26274.
- Pritam Dash et al. From untrusted input to trusted memory: A systematic study of memory poisoning attacks in LLM agents, 2026. arXiv:2606.04329.
- Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. MINJA: Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2503.03704.
- Stephanie Forrest, Steven A Hofmeyr, Anil Somayaji, and Thomas A Longstaff. A sense of self for Unix processes. In *Proceedings of the IEEE Symposium on Security and Privacy*, 1996.
- Diego Garcia-soto, Huili Chen, and Farinaz Koushanfar. PerD: Perturbation sensitivity-based neural trojan detection framework on NLP applications, 2022. arXiv:2208.04943.
- Ishrith Gowda. MEMSAD: Gradient-coupled anomaly detection for memory poisoning in retrieval-augmented agents, 2026. arXiv:2605.03482.
- Jun Wen Leong. Injection-execution dissociation: A mechanistic evaluation of persistent memory attacks and defenses in stateful LLM agents. 2026a. arXiv:2605.08442.
- Jun Wen Leong. Recognition without enforcement: Configuration-dependent failures in LLM agent instruction arbitration and external control, 2026b. arXiv:2608.28502.
- Ying Li et al. VIGIL: Runtime enforcement of behavioral specifications in AI agent skills, 2026. arXiv:2606.26524.
- Jiahao Liu et al. TraceAegis: Securing LLM-based agents via hierarchical and behavioral anomaly detection, 2025. arXiv:2510.11203.
- Ciyan Ouyang and Rui Hou. MemLineage: Lineage-guided enforcement for LLM agent memory, 2026. arXiv:2605.14421.
- Sidharth Pulipaka et al. Hidden in memory: Sleeper memory poisoning in LLM agents, 2026. arXiv:2605.15338.
- Zijun Wang et al. Proactive safety auditing for multi-turn LLM agents via trajectory-state modeling, 2025. arXiv:2605.27690.
- Christina Warrender, Stephanie Forrest, and Barak Pearlmutter. Detecting intrusions using system calls: Alternative data models. In *Proceedings of the IEEE Symposium on Security and Privacy*, 1999.
- Yuanbo Xie et al. What if prompt injection never left? exploring cross-session stored prompt injection in agentic systems, 2026. arXiv:2606.04425.
- Xianglin Yang, Yufei He, Shuo Ji, Bryan Hooi, and Jin Song Dong. Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections, 2026. arXiv:2602.15654.
- Xuanye Zhang et al. MemMorph: Tool hijacking via memory poisoning, 2026. arXiv:2605.26154.